ORACLE University

Developing Web Applications with JavaScript, HTML5, and CSS

Electronic Presentation

D1104149GC10

Learn more from Oracle University at **education.oracle.com**

1001162023

ORACLE

Developing Web Applications with JavaScript, HTML5, and CSS

Course Goals

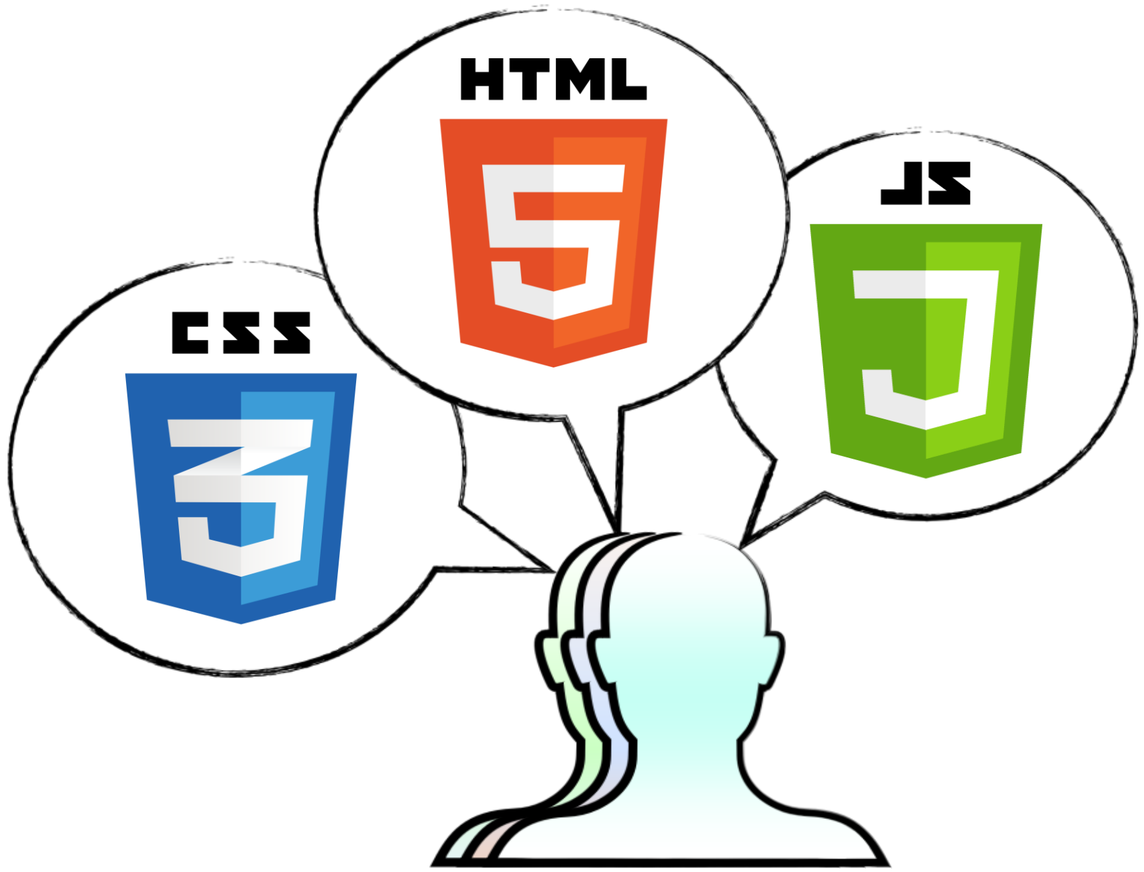
In this course, you will learn to:

- Design application User Interface (UI) with Hyper Text Markup Language (HTML)
- Enhance visual appearance of the UI with Cascading Style Sheets (CSS)
- Implement client-side application logic with JavaScript
- Invoke REST Services
- Interact with WebSockets

Audience

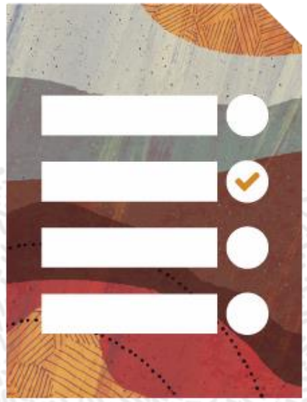
The target audience includes those who want to learn :

- JavaScript
- HTML
- CSS

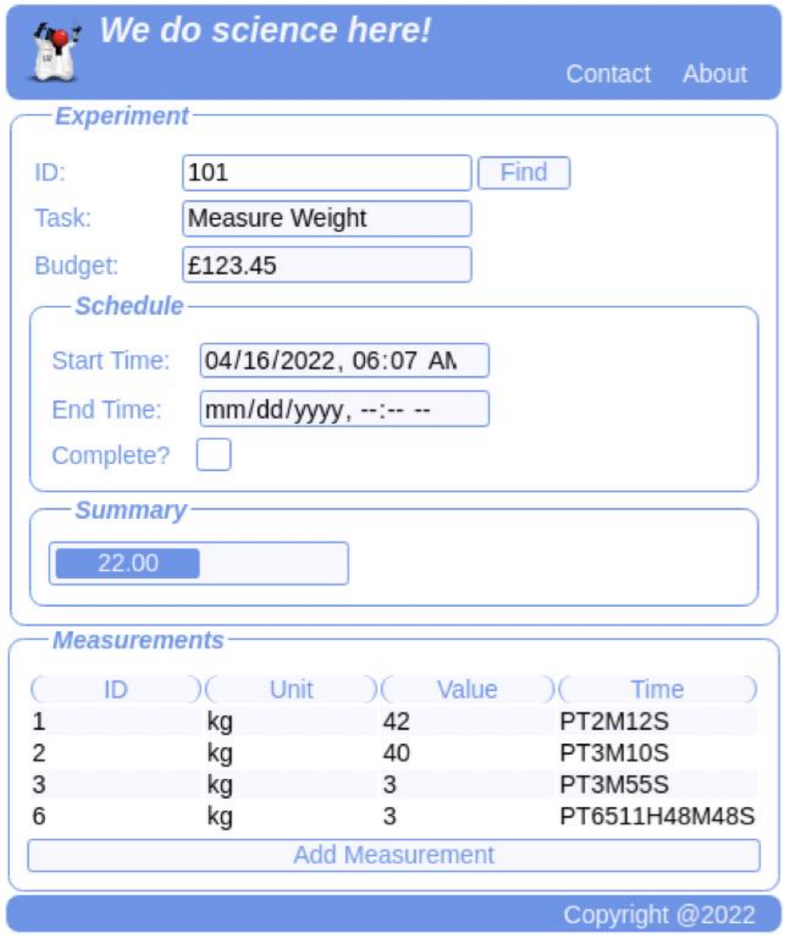


Course Outline

- Introduction to Web Applications
- Introduction to JavaScript
- JavaScript Functions, Classes, and Objects
- Extending JavaScript Classes and Objects
- Using JavaScript in a Browser Environment
- Creating Cascading Style Sheets
- Advanced JavaScript
- Interact with REST and WebSocket Servers
- Overview of JavaScript APIs, Frameworks, and Variants



Course Practices



During the course practice sessions, you will:

- Explore the features of JavaScript, HTML, and CSS.
- Develop a Web Application to track scientific experiments and record measurements.

ORACLE

1

Introduction to Web Applications

Objectives

After completing this lesson, you should be able to:

- Define what a web application is
- Describe how HTTP is used
- Describe requests, response, and errors codes using in HTTP interactions
- Describe HTML syntax and document structure
- Associate the page with resources, such as images, styles, and scripts
- Add simple JavaScript code to an HTML page
- Submit data and perform navigation

Introduction to Web Application

Is a client/server application that is deployed on web servers

Is accessed over the Internet using web clients such as browsers

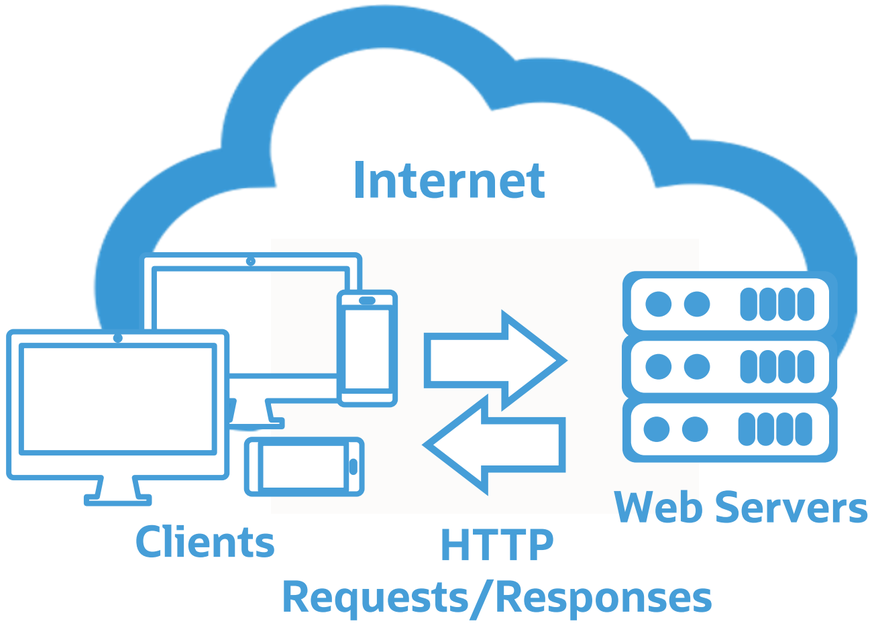
Displays documents on the browsers, where the content type can be:

- Static
- Dynamic

Displays web content as documents that include:

- Document pages
- Images
- Stylesheets
- Scripts, etc.

Is accessed using Hypertext Transfer Protocol (HTTP)

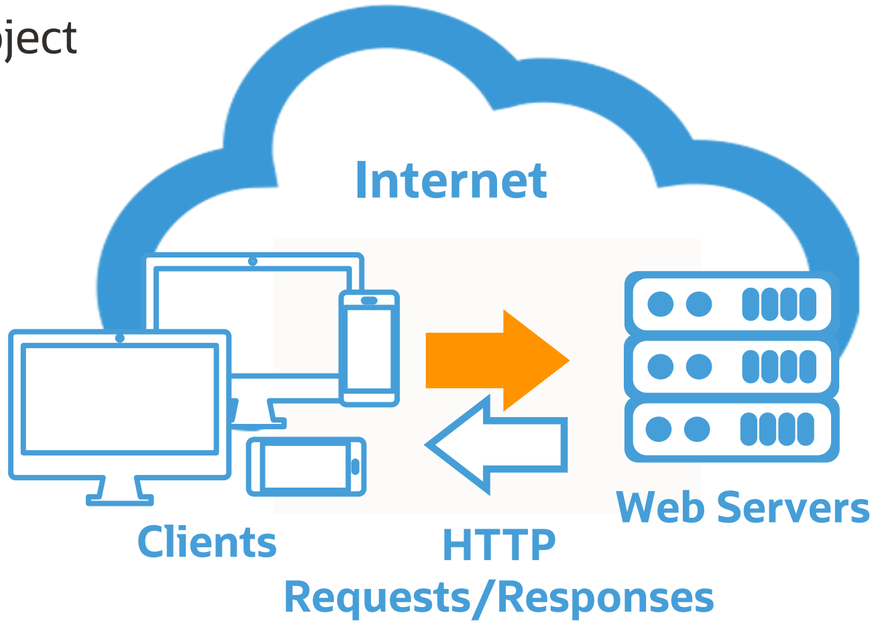


HTTP Requests Methods

HTTP Requests are dispatched using methods, such as:

- GET - Retrieve headers and content from the server
- HEAD - Retrieve headers only from the server
- POST - Upload data to the server, to create new objects
- PUT - Upload data to the server, to modify existing objects
- PATCH – Upload data to the server, to modify parts of the object
- DELETE - Instruct server to delete information
- CONNECT - Establish connection to the server
- TRACE - Ask server to return each request back
- OPTIONS - Identify which methods are supported

Note: This is not an exhaustive list of methods.



HTTP GET and HEAD Methods

GET and **HEAD requests** only contain packet headers and no body.

- Parameters are transmitted as part of the request URL header.
- They are typically used to request initial pages where parameters may not be required.

```
GET /labs/experiment?id=101&startDate=2020-08-01 HTTP/1.1
Host: dukelabs.org
```

- Successful **GET responses** are expected to contain packet header and body.
- Successful **HEAD responses** only contain packet headers and no body.

```
HTTP/1.1 200 OK
Date: Tue, 11 Aug 2020 22:50:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 150
<!DOCTYPE html>
<html>
  // some page content
</html>
```

HTTP POST Method

POST requests contain both packet headers and body.

- Parameters are transmitted as part of the request body.
- They are typically used to submit data, for example, to instruct the server to create objects.

```
POST /labs/experiment HTTP/1.1
Host: dukelabs.org
id=101&startDate=2020-08-01
```

- Successful **POST responses** are expected to contain packet header and body.

```
HTTP/1.1 200 OK
Date: Tue, 11 Aug 2020 22:50:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 150
<!DOCTYPE html>
<html>
  // confirmation of the resource creation
</html>
```

HTTP PUT and PATCH Methods

PUT or **PATCH requests** contain both packet headers and body.

- Values are transmitted as part of the request body.
- Object identity is transmitted as part of the URL header.
- It is typically used to submit data, for example, to instruct the server to update an object.

```
PUT /labs/experiment/101
&startDate=2020-08-01
```

- Successful **PUT** or **PATCH responses** are expected to contain packet header and body.

```
HTTP/1.1 200 OK
Date: Tue, 11 Aug 2020 22:50:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 150
<!DOCTYPE html>
<html>
 // confirmation of the resource update
</html>
```

HTTP DELETE Method

DELETE requests contain packet headers and may contain a body.

- Values are transmitted as part of the request body.
- Object identity is transmitted as part of the URL header.
- It is typically used to submit data, for example, to instruct server to update an object.

```
DELETE /labs/experiment/101
```

- Successful **DELETE responses** are expected to contain packet header and may contain a body.
- Response status code indicates success or failure and the presence or absence of the response body.

```
HTTP/1.1 200 OK
Date: Tue, 11 Aug 2020 22:50:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 150
<!DOCTYPE html>
<html>
 // confirmation of the resource deletion
</html>
```


HTTP Responses and Status Codes

HTTP Headers

- Context Type
- Content Length
- Character Encoding
- Date and Time
- HTTP Status Code
  - 1xx codes: Informational
  - 2xx codes: Successful Responses
  - 3xx codes: Redirection
  - 4xx codes: Client Errors
  - 5xx codes: Server Errors

```
HTTP/1.1 200 OK
Date: Tue, 11 Aug 2020 22:50:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 150
<!DOCTYPE html>
<html>
  // content
</html>
```

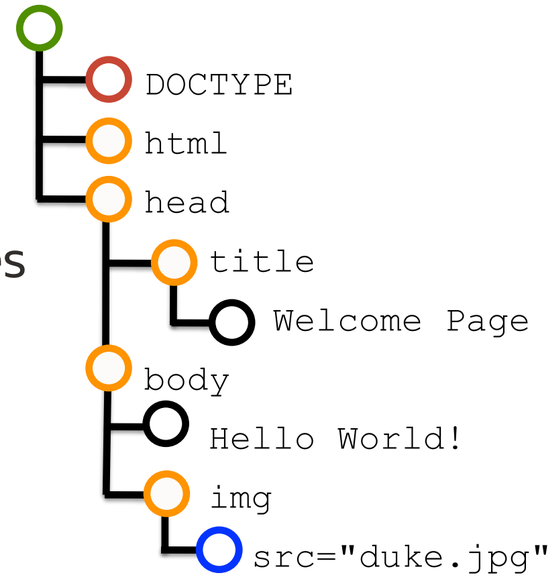
HTTP Body (optional)

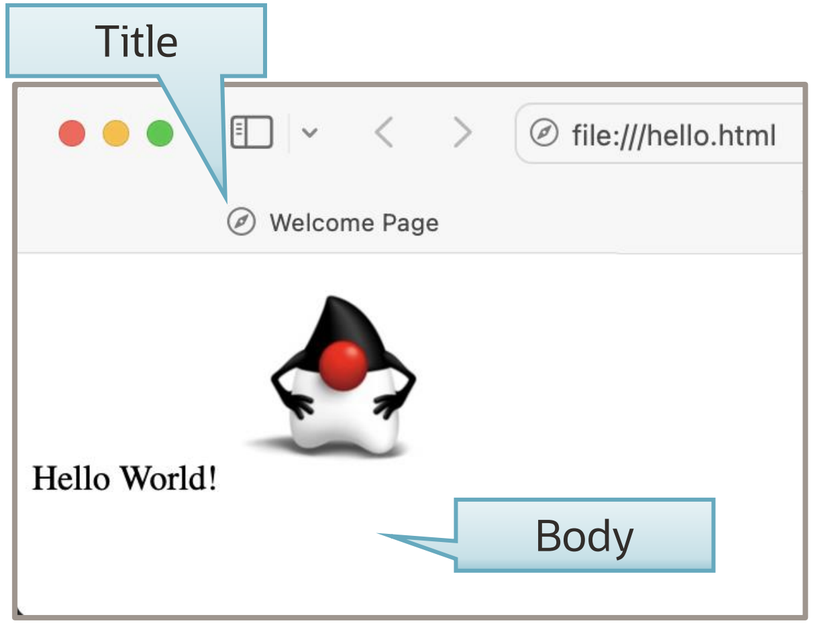
- Any data, such as images, HTML pages, JSON or XML messages

HTML Document Object Model (DOM)

Is a standard object model and programming interface for HTML
On loading a web page, the browser creates DOM of this page
The HTML Document Model for the page include different types of nodes

- Document Root
- Declarations
- Processing Instructions
- Elements
- Attributes
- Text
- Comments






```
<!DOCTYPE html>
<html>
  <head>
    <title>Welcome Page</title>
  </head>
  <body>
    Hello World!
    <img src="duke.jpg">
  </body>
</html>
```

HTML Document Page Layout

Logical (semantic) markups define page logical structure:

- Content is structured as articles and sections.
- Additional structural units include header, footer, navigation, aside the elements.

Physical markups define elements with specific functionality and appearance:

- Input elements, URL links, buttons, images, tables, lists, and so on …

```
<body>
  <section>
    <nav>navigation item</nav>
    <header>Section Header</header>
      Content of a section
      <aside>
        <img src="duke.jpg">
      </aside>
    <footer>other details</footer>
  </section>
</body>
```

HTML Tables

Organize layout as rows and columns

- Markup `<tr></tr>` encloses each row.
- Markups `<th></th>` and `<td></td>` enclose table cells that contain headers or values.

```
 <table border='1' width='30%'>
  <caption>Table Title</caption>
    <tr>
      <th width='60%'>Column 1 Header</th>
      <th width='40%'>Column 2 Header</th>
    </tr>
    <tr>
      <td>Row 1 Column 1 Value</td>
      <td rowspan="2">Rows 1 and 2 Column 2 Value</td>
    </tr>
    <tr>
      <td>Row 2 Column 1 Value</td>
    </tr>
    <tr>
      <td colspan="2">Row 3 Columns 1 and 2 Value</td>
    </tr>
</table>
```

Table Title

Column 1 Header	Column 2 Header
Row 1 Column 1 Value	Rows 1 and 2 Column 2 Value
Row 2 Column 1 Value	
Row 3 Columns 1 and 2 Value	

HTML Lists

Organize layout as lists

- Markup `<ul></ul>` encloses unordered and `<ol></ol>` encloses ordered lists.
- Markup `<li></li>` encloses list items.

```
<ul>
 <li>Item 1</li>
 <ul style='padding-left:10px;'>
   <li>Item 1.1</li>
   <li>Item 1.2</li>
 </ul>
 <li>Item 2</li>
</ul>
<ol start='3'>
 <li>Item 1</li>
 <ol type='a'>
   <li>Item 1.1</li>
   <li>Item 1.2</li>
 </ol>
 <li>Item 2</li>
</ol>
```

- Item 1
  - Item 1.1
  - Item 1.2
- Item 2

3. Item 1
    a. Item 1.1
    b. Item 1.2
4. Item 2

Add Style to HTML Elements

Cascading Style Sheet (CSS) describes visual styles for HTML elements.

- A style can be applied to all matching elements across the entire document.
- A style can be overridden for a specific element.
- It controls colors, fonts, sizing, alignment, and other properties.


```
<html>
  <head>
    <style>
      section {
        color:red;
        text-align:center;
      }
    </style>
  </head>
  <body>
    <section>
      Section A
    </section>
    <section style="color:blue;">
      Section B
    </section>
  </body>
</html>
```

Add JavaScript to HTML Document

JavaScript allows code execution on the client.

```
<html>
  <head>
    <script type="text/javascript">
      console.log("Hello!");
    </script>
  </head>
  <body>
  </body>
</html>
```

Associating Style Sheet with HTML Document

Adding CSS File reference to an HTML Page

hello.html

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <section>
      Section A
    </section>
  </body>
</html>
```

style.css

```
section {
  color:red;
  text-align:center;
}
```

Associating JavaScript with HTML Document Page

Adding JavaScript File reference to an HTML Page

hello.html

```
<<!DOCTYPE html>
<html>
  <head>
    <script src="hello.js"></script>
  </head>
  <body>
  </body>
</html>
```

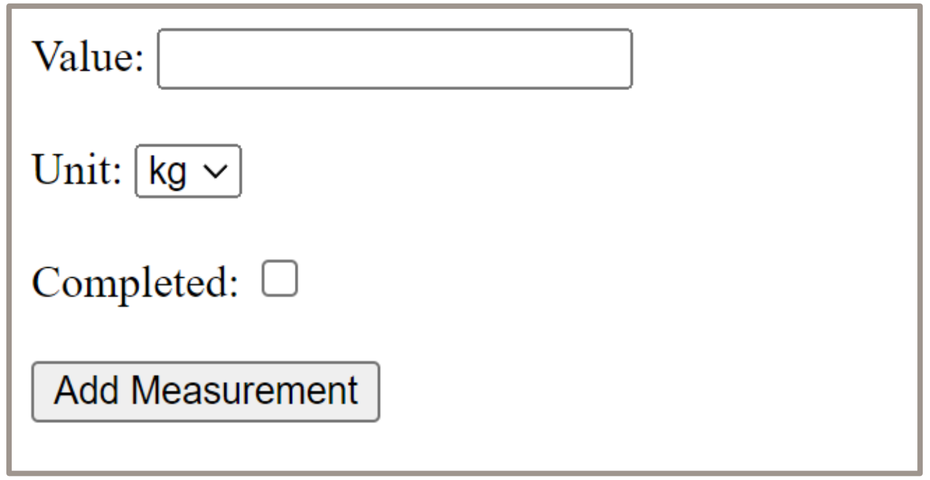
hello.js

```
console.log("Hello!");
```


HTML Forms

Form allows client to submit values.

- Attribute action defines a server URL that will handle submitted values.
- Attribute method defines an HTTP method, such as GET, POST, PUT, PATCH, DELETE.
- Elements such as select and input define data input controls.
- Input controls act as parameters to be submitted to the server.
- Each parameter is identified by the name attribute.
- Input type attribute describes different visual controls.




```
<form action='server url' method='POST'>
  Value: <input type='text' name='value'>
  Unit:
  <select name='unit'>
    <option value='Kilogram'>kg</option>
    <option value='Second'>s</option>
  </select>
  Completed:
  <input type='checkbox' name='completed'>
  <input type='submit' value='Add Measurement'>
</form>
```

Implementing Navigation in an HTML Document

A clickable link (anchor) `<a>` element defines a navigation target using `href` attribute.

```
<a href="target url">Send email</a>
```

Submitting a form defined by a `<form>` element may result in navigation.

- Attribute `action` defines a server URL that handles submitted form data.
- Action URL may return a new document as a result of form data processing.

```
<form action='target url' method='POST'>
 // form components
</form>
```

- Navigation target can be any page element.

```
<nav>
  <a href="#x">Section X</a>
  <a href="#y">Section Y</a>
</nav>
<section id="x">...</section>
<section id="y">...</section>
```

Summary

In this lesson, you should have learned how to:

- Define and describe web applications and HTTP
- Describe HTML syntax and document structure
- Associate page with resources, such as images, styles, and scripts
- Submit data and perform navigation

Practice Overview

This practice covers the following topics:

- 1-1: Produce Web Application User Interface (UI)
- 1-2: Test and inspect Web Application

ORACLE

2

Introduction to JavaScript

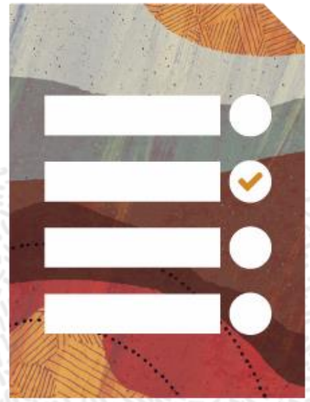
Objectives

After completing this lesson, you should be able to:

- Describe the justification for JavaScript
- Declare and initialize variables
- Use JavaScript data type system
- Perform operations with Text, Numeric, Boolean, and Date values
- Explain value conversions and type casting
- Introduce arrays and collections
- Implement control flow with `if/else`, switch, and loop constructs

Agenda

- Introduction to JavaScript
- JavaScript Syntax
- Introduction to Operators
- Introduction to Arrays
- Control Flow



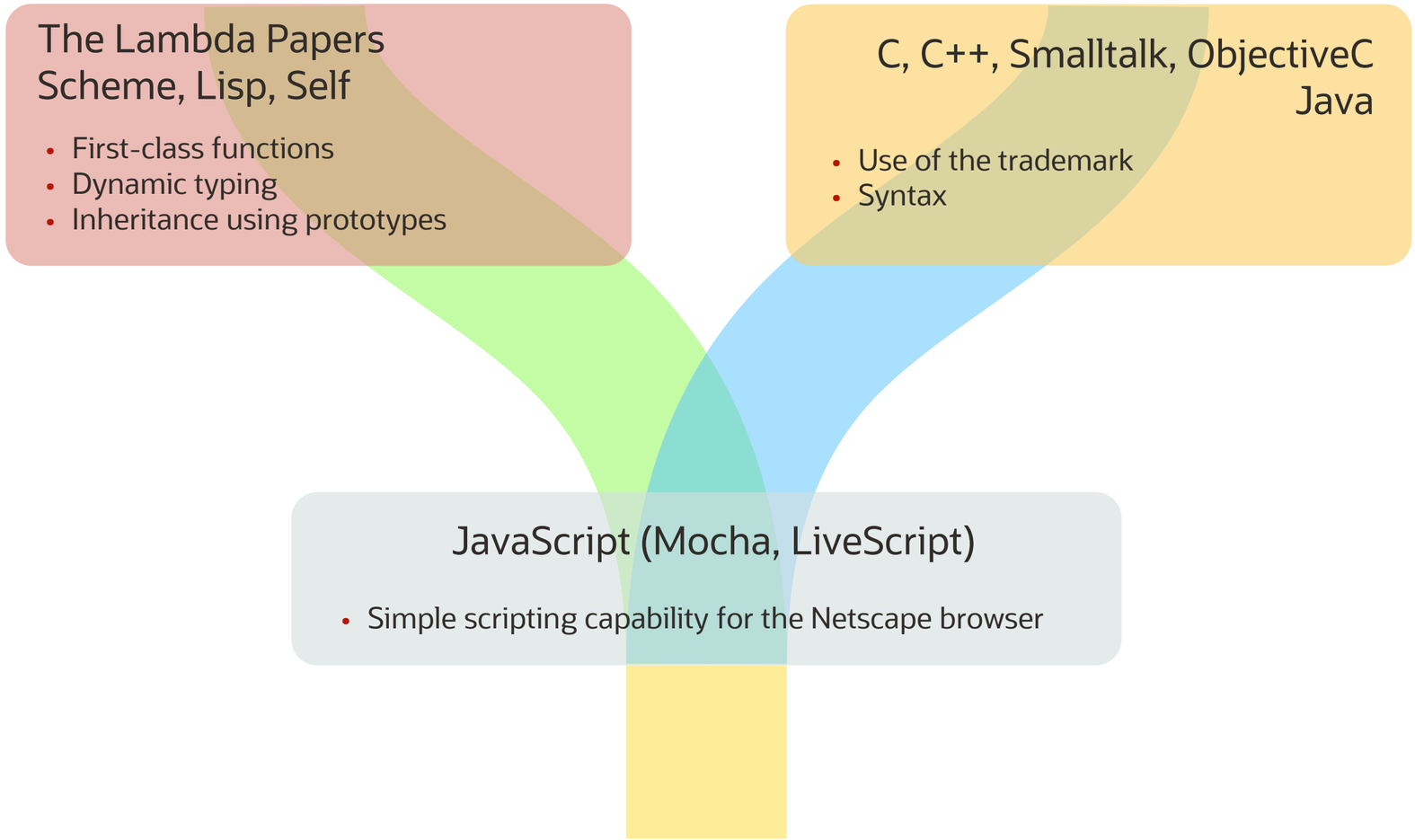
Introduction to JavaScript

- Mixed, functional, and Object-oriented
- Interpreted Language
- Officially maintained by ECMA as ECMAScript
- Used for both Server-side and Client-side scripting

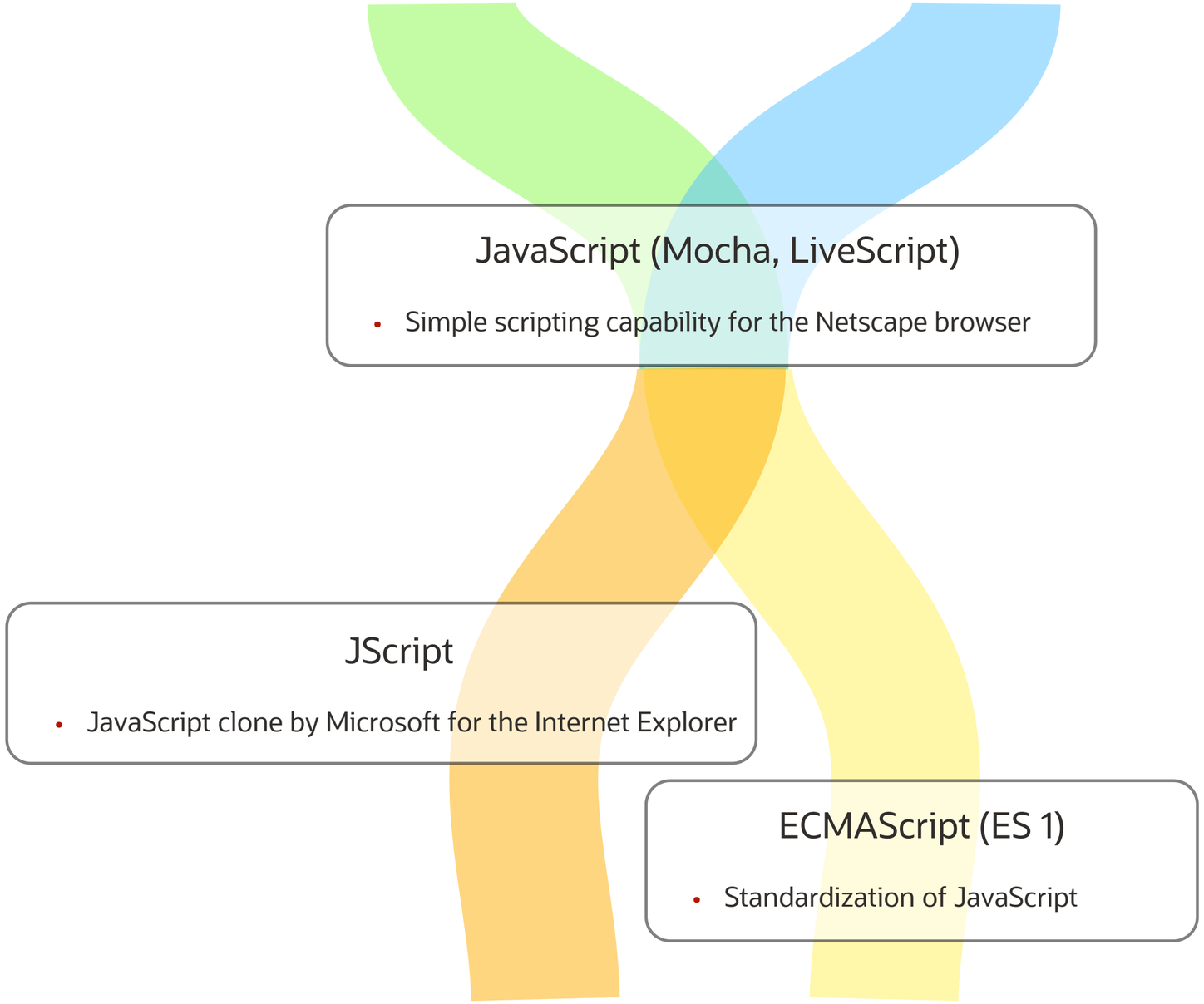

```
// Client Side JavaScript with HTML Document
<html>
 <head>
 <title>JS Inline Script in HTML Document Page</title>
</head>
<body>
     //Embedded JavaScript
    <button onclick= "alert('JavaScript!')">Click here</button>
</body>
</html>
```

✤ This course focuses on the core JavaScript as used within the browser environment.

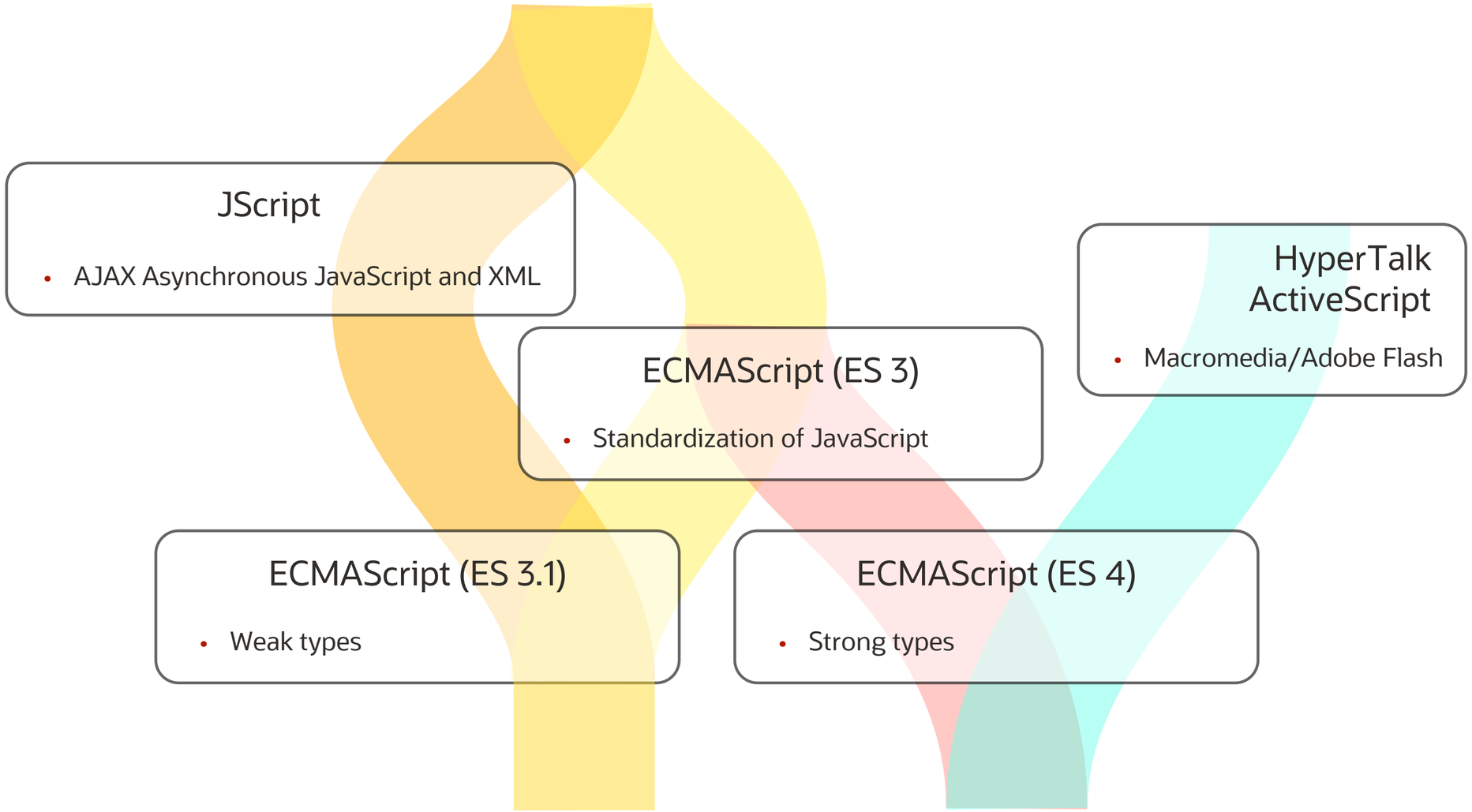
JavaScript Origins



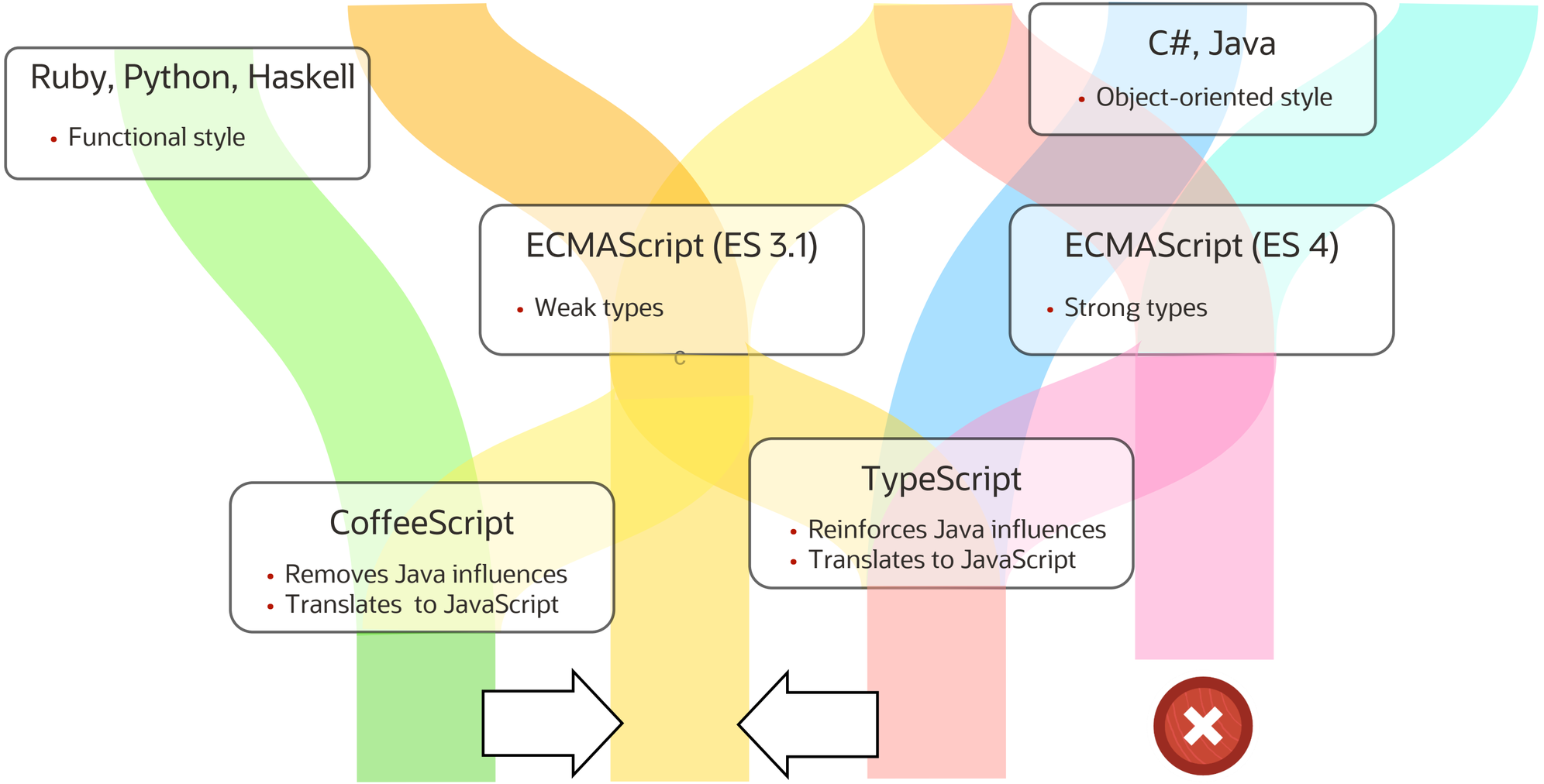
JavaScript Clones and Standards



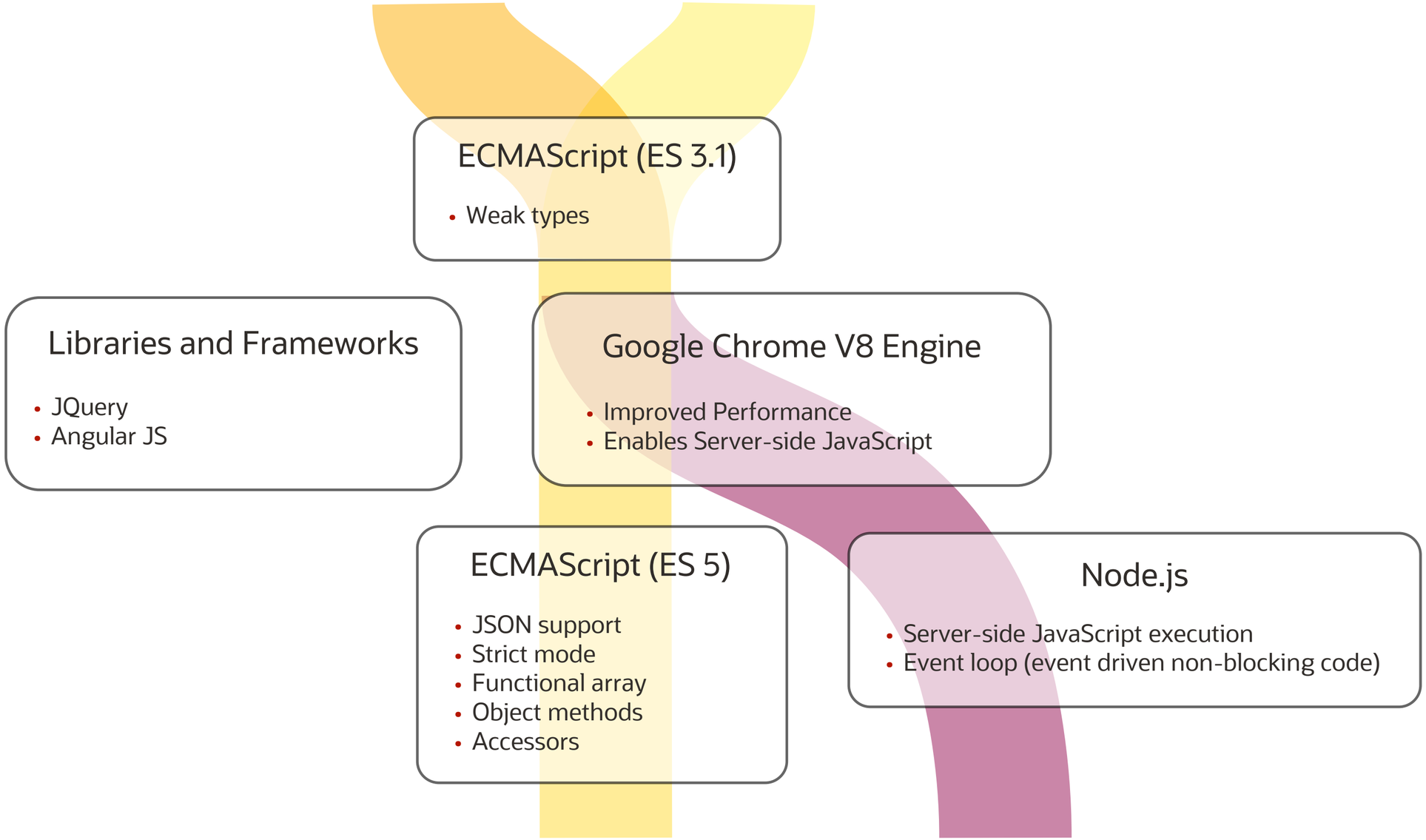
JavaScript Standards Split



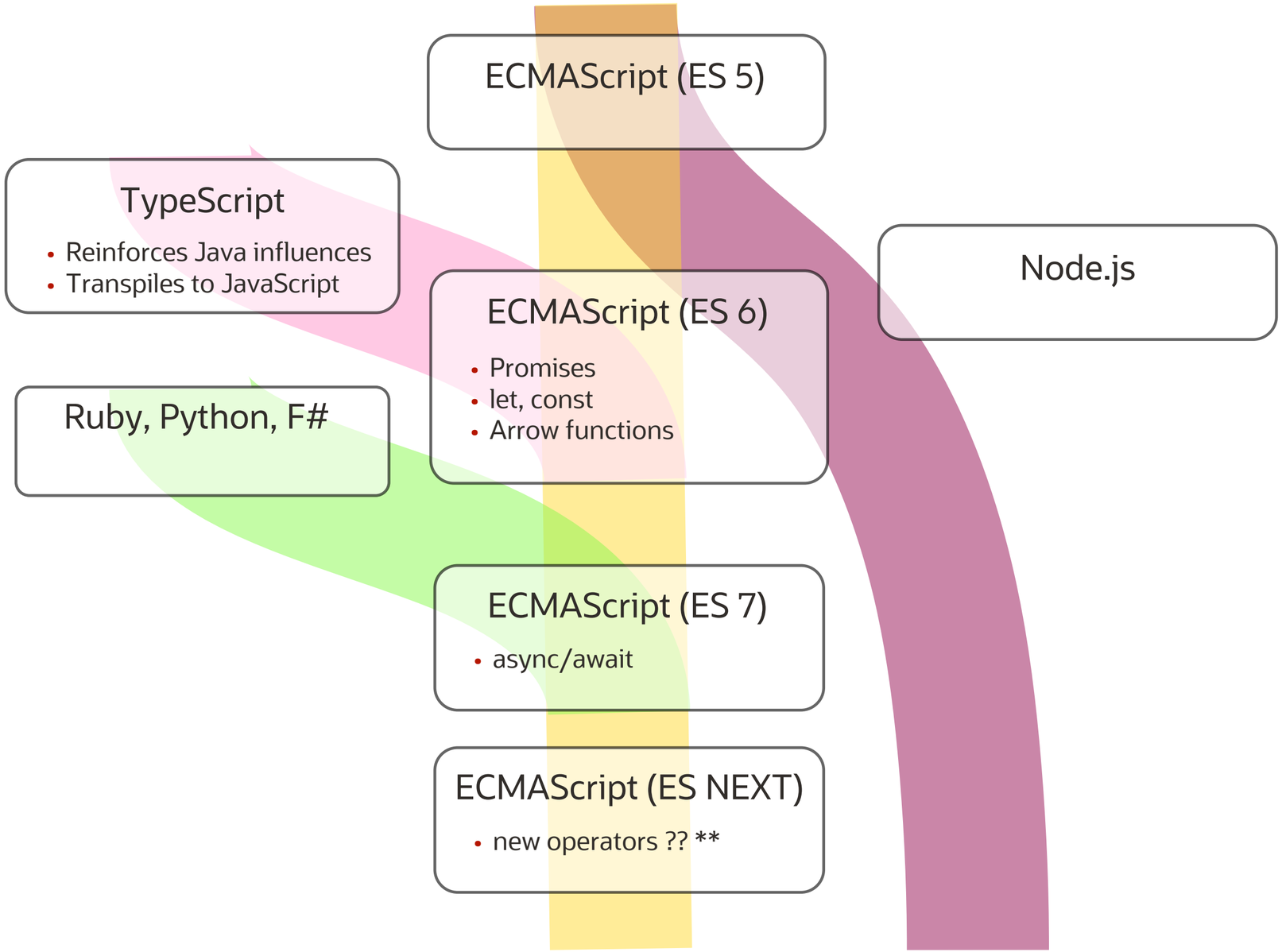
JavaScript Variants Proliferation



JavaScript New Engine and Server-Side Expansion

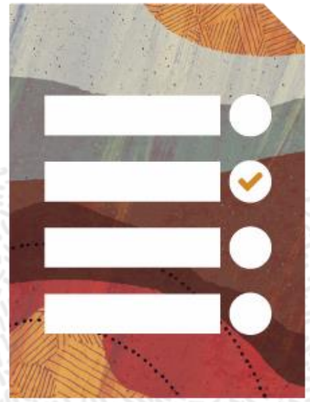


JavaScript Modern Evolution



Agenda

- Introduction to JavaScript
- JavaScript Syntax
- Introduction to Operators
- Introduction to Arrays
- Control Flow



JavaScript Syntax

A set of rules that illustrates how to write a JavaScript Program

- Statements should be terminated with the semicolon `;`
- Blocks of code must be enclosed within braces `{ }`
- String literals are enclosed within:
  - Single `' '` or
  - Double `" "` quotes
- Comments are added to the code using:
  - Multiline `/* */`
  - Single line `/ /`
- Functions are defined using:
  - `function` keyword
  - Name of the function
  - Parentheses ( ) - parameters separated by commas can be included

```
/* Multiline
   comment.  */
'use strict';

let x = "hello", y = 'world';

function hello() {
   // single line comment
}
```

Additional code verification checks are enabled using the `strict mode` directive.

JavaScript Variables

Variable Declarations

- `var` is visible within the entire function
- `let` (recommended) is visible within specific block and cannot be redeclared
- `const` defines constants
- Global scoped variables are *"Hoisted"*

Variable Initializations

- Values are assigned using = assignment operator

```
a = "hello";
var a;
const b = 42;
b = 22;
function acme() {
  if(true) {
    c = 3;
    var c = 1;
    var c = 2;
    let d = 1;
    let d = 2;
  }
  console.log(c);
  console.log(d);
}
console.log(c);
console.log(d);
```

JavaScript Data Types

JavaScript defines with the following data types:

- Primitive: `string`, `number`, `bigint`, `boolean`, `symbol`, `undefined`, `null`
- Complex: `object` (including array), `function`

Identifying variable types

- JavaScript types are weak.
  - A variable can be assigned values of different types.
  - Variable type is dynamically determined based on what the variable value is at any given moment.
- `typeof` operator gets the datatype of a variable or an operand.
- `instanceof` operator checks if a value is of a certain type.

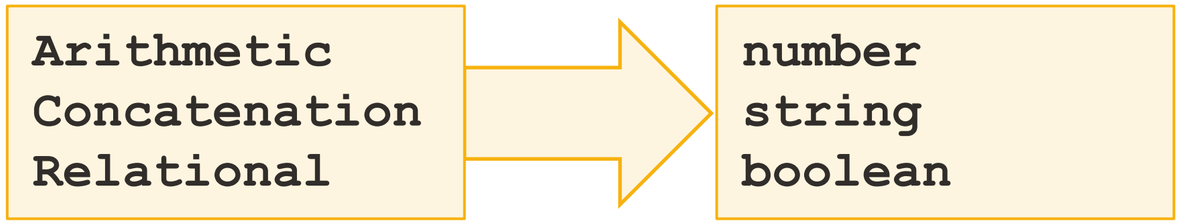
```
let x;
x = null;
x = 5;
x = BigInt(5);
x = "acme";
x = true;
x = Symbol("acme");
x = function(){};
x = new Date();
x = {};
x = [1,2,3];
(typeof x);
(x instanceof number);
```

❖ **Note:** Primitive values are immutable.

JavaScript Types Coercion

JavaScript attempts to automatically convert types based on a context in which they are used.

- Attempts to convert to `number` in a context of arithmetic operations
- Attempts to convert to `string` in a context of a concatenation operation
- Attempts to convert to `boolean` in a context of relational or equality operations




```
let x1 = "1"+1;              // number 2
let x2 = "a"+1;              // string a1
let x3 = true+2;             // number 3
let x4 = !"a";               // boolean false
```

❖ **Note:** Type coercion may be a cause of bugs.

Explicit Value Conversions

- `toString()` and `String()` converts value to string primitive.
- `parseFloat()`,`parseInt()`, and `Number()` converts value to number primitive.
- `toFixed()`,`toPrecision()`, and `String()` converts number value to string primitive.
- `Boolean()` converts value to boolean primitive.
- `new String()` converts value to String object.
- `new Number()` converts value to a Number object.
- `new Boolean()` converts value to a Boolean object.

```
let v1 = 12.99, v2="12.99", v3;
v3 = v1.toString();      // "12.99" as string
v3 = String(v1);         // "12.99" as string
v3 = new String(v1);     // "12.99" as object
v3 = parseFloat(v2);     // 12.99 as number
v3 = parseInt(v2);       // 12 as number
v3 = parseInt("2E");     // NaN as number
v3 = parseInt("2E",16); // hex 46 as number
v3 = Number(v2);         // 12.99 as number
v3 = new Number(v2);     // 12.99 as object
v3 = v1.toFixed(3);      // 12.990 as string
v3 = v1.toFixed(1);      // 13.0 as string
v3 = v1.toPrecision(5); // 12.990 as string
v3 = v1.toPrecision(3); // 13.0 as string
v3 = Boolean(v1);        // true as boolean
v3 = Boolean("false");   // true as boolean
v3 = new Boolean(0);     // false as object
```

JavaScript Objects and Functions

Object is a complex data type containing key-value pairs.

- Value can be a simple value, another object, or a function.
- Values can be accessed with a dot notation.

Function is a block of code designed to perform actions.

- May accept parameters
- May return a value

```
let weight = {
  unit: "kg",
  value: 42,
  getLb() {
    return this.value*2.20462;
  }
};
weight.value = 20;
let lb = weight.getLb();
```

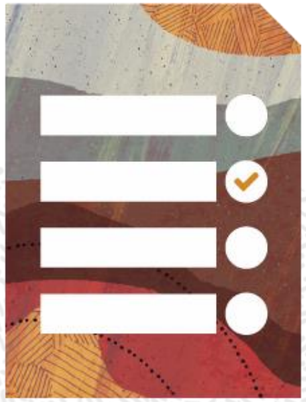
```
function kgToLb(kg) {
  return kg*2.20462;
};
let lb = kgToLb(20);
```

Object properties syntax rules:

- Colon : separates property name and value
- Comma , separates properties

Agenda

- Introduction to JavaScript
- JavaScript Syntax
- Introduction to Operators
- Introduction to Arrays
- Control Flow



JavaScript Operators

Operators work on one or more values, called *operands*, to yield a single result.

- Arithmetic (`+ - * / % ++ -- **`)
- Assignment (`=`)
- Comparison (`== === != !== < > >= <=`)
- Boolean (`&& || !`)
- Bitwise (`& | ^ ~ >>> >> <<`)
- Ternary (`?: ?? ?.`)
- Object (`typeof instanceof new new.target in delete super this`)
- Other (`yield void , . ...`)

```
let x = 5, y = 4, z;
z = x + y;      // z is 9
z = x++;        // x is 6 z is 5
z = ++y;        // z and y are 5
z = 5;
z *= 2;         // z is 10
z = (x == y); // z is false
```

Notes:

- Compound assignment combines an operator with assignment, such as: `+= -= *= /= &=`
- Arithmetic addition `+` operator also works with strings for concatenation

Manipulate Numeric Values Using Arithmetic Operators

Arithmetic operator precedence order

1. `()` enclose expressions that are evaluated first.
2. `- + ++ --` unary operators, increment and decrement
3. `* / %` multiplication/division
4. `+ -` addition/subtraction
5. `= += -= *= /= %= **=` assignment and compound assignment

```
let x = 25, y = 20, z = 0;
z = ((x + y) * x) * y/x ;    // 900
z =   x + y  * x  * y/x ;    // 425
z = 1/0;                     // Infinity
z = 1*"a";                   // NaN
```

Notes:

- Floating point math is not correctly implemented in JavaScript. Values need to be converted to integers before arithmetic operations, and only the results are converted to floating point.
- Numbers that exceed 15 digits overflow.

Extract Portions of Text from a String

- String characters are indexed starting from `0`.
- Property `length` indicates a total number of characters.
- Extract portions of the string using:
  - `substring(from_index, to_index)` the `to_index` is not inclusive
  - `slice(from_index, to_index)` allows the negative index to start at the end of the string
  - `substr(from_index, length)`

```
//        0123456   index representation
let s = "abcdefg";
s.length;                                          // 7
s.substring(2,4)+" "+s.substring(2);                 // cd cdefg
s.slice(2,4)+" "+s.slice(2)+" "+s.slice(-4,-2); // cd cdefg de
s.substr(2,4)+" "+s.substr(2);                       // cdef cdefg
```

❖ **Note:** The `to_index` and `length` parameters are optional - the remainder of the string is taken when they are not present.

Concatenate Strings

Concatenate Strings using:

- `string1.concat(string2)` function
- `+` and `+=` operators

```
let a = "u", b = 2, c = 2, d = "2";
let e = a+b+c; // u22
let f = b+c+a; // 4u
a += b;        // u2   equivalent to  a = a + b;
a.concat(b);   // u22  string a is not reassigned, i.e., a remained "u2"
let g1 = d+b;  // 22   number is coerced to string
let g2 = d*b;  // 4    string is coerced to number
```

❖ **Note:** Operator `+` is also used as an arithmetic addition, so the order and the nature of values matter.

Manipulate Text Values

Remove leading and trailing spaces:

- `trim()`

Detect if text contains a certain string value:

- `indexOf(string) lastIndexOf(string)`
- `startsWith(string) endsWith(string) includes(string)`

Convert string case:

- `toUpperCase() toLowerCase()`

```
//       012345   index representation
let s = " abcd ";
s = s.trim();          // "abcd"
s = s.toUpperCase(); // "ABCD"
s.indexOf("B");        // 1
s.includes("CD");     // true
s.startsWith("AB");   // true
```


Compare Values

Relational and equality operators are used to compare values.

- Equals `==` and strictly equals `===`
- Not equals `!=` and not strictly equals `!==`
- Less than, greater than, less than or equal, greater than or equal: `< > >= <=`
- Conditions may be combined using and `&&` or `||` exclusive or `^` operators

The result of the comparison is always a Boolean value (either `true` or `false`).

```
let x1 = 2, x2 = "2", x3;
x3 = (x1 == x2);              // true
x3 = (x1 === x2);             // false
x3 = (x1 === +x2);            // true
x3 = (x1 > x2 || x1 != x2); // false
x3 = !false;                  // true
x3 = new Boolean(true);       // true
x3 = new Boolean(0);          // false
x3 = new Boolean(-42);        // true
```

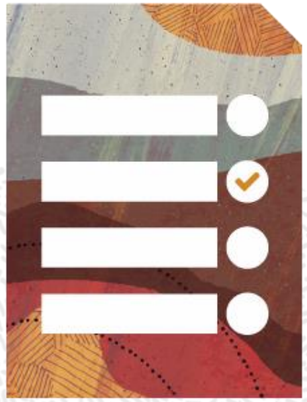
Manipulate Date Values

- Create Date Objects
- Modify Date Objects
- Acquire specific values from Date Objects and calculate periods of time
- Format Date Objects as text

```
let a = new Date();
let b = new Date(2022, 4, 1);
let c = new Date(1961,3,12,6,7);
a.setMonth(3);
a.setHours(10);
let elapsed = a.getTime() - c.getTime(); // 1924055580000 milliseconds
c.getHours(); // 6
c.getMinutes(); // 7
c.getMonth(); // 3
c.toLocaleString("en-GB", {timeZone: "Europe/London"}); // 12/04/1961, 06:07:00
```

Agenda

- Introduction to JavaScript
- JavaScript Syntax
- Introduction to Operators
- Introduction to Arrays
- Control Flow



Manipulate Arrays

- Arrays are complex variables that store a number of values.
- Declare and initialize an array.
- Access an element in an array.
- Get the length of an array.

```
let values = [1,2,3];
values = [1,2,3,"10","12"];
values[3] = "a";
let x = values[2]; // Value of x is 3
let size = values.length; // Size is 5
x = values[size-1]; // Value of x is "12"
```

Complex Arrays and Collections

Multidimensional arrays

- Each group of `[]` indicates a “dimension”.
- Access elements using indexes for each dimension.

	0	1
0	a	b
1	c	d

```
let values = [['a','b'],['c','d']];
let value = values[1][0];
```

Other collections include:

- `Set` – a collection of unique elements
- `Map` – a key-value pair collection

```
let values = new Set();
values.add('a');     // add 'a'
values.add('b');     // add 'b'
values.add('a');     // add nothing
values.has('a');     // true
values.delete('a'); // remove 'a'
values.size;         // 1
let array = [1,2,3];
values = new Set(array);
```

```
let values = new Map();
values.set('a',2);  // add 'a',2
values.set('a',5);  // replace 'a',5
values.set('b',4);  // add 'b',4
values.delete('a'); // remove 'a'
values.size;        // 1
```

Array Object

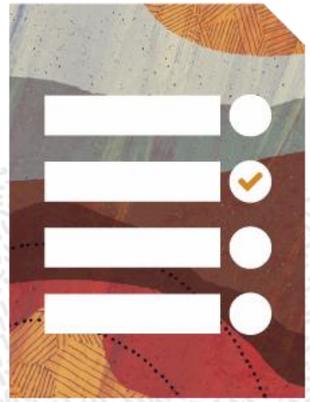
Provides useful operations to manipulate arrays, such as:

- Convert collections into arrays
- Fill and copy array elements
- Search and sort elements
- Iterate through array content

```
let values = new Set();
values.add('a');     // add 'a'
values.add('b');     // add 'b'
values.add('c');     // add 'c'
// create array from a collection
let array = Array.from(values);
// creates array from a variable number of arguments
array = Array.of(42,15,71);
// return true if the argument is an array
Array.isArray(array);
// find index of an element
let x = array.indexOf(12); // -1
// order array elements
array.sort((e1,e2) => e2-e1); // 71 42 15
```

Agenda

- Introduction to JavaScript
- JavaScript Syntax
- Introduction to Operators
- Introduction to Arrays
- Control Flow



Implement Flow Control with `if /else`

- Execute `if` or `else` block of code depending on the boolean condition expression.
- The `else` clause is optional.
- Block delimiters can be omitted when executing a single statement.
- Another `if/else` block can be embedded inside the `if` or `else` block.

```
if(condition expression){
  // block of code executed when condition is true
} else {
  // block of code executed when condition is false
  if (condition expression) {

  } else {

  }
}
```

```
if(condition expression)
  // single statement
else
  if (condition expression)
    // single statement
  else
    // single statement
```

Ternary Operations

- Ternary `?:` (conditional assignment) is an `if/else` alternative

```
variable = (condition expression)
   ? value when condition is true : value when condition is false;
```

- Nullish coalescing operator `??` substitutes `null` or `undefined` values

```
let x = 2, y =3, z;
if (x > y) {
  z = "a";
} else {
  z = "b";
}
```

```
let x = 2, y =3, z;
z = (x > y) ? "a" : "b";
```

```
let x, z;
z = (x == null || x == undefined)
  ? "a" : x;
z = x ?? "a";
z = x || "a";
```

- Or || operator returns right hand operand when if the left operand is false.

Implement Flow Control with Switch

- The switch statement evaluates the **expression** and navigates to the case that matches it.

```
switch(expression) {
  case 1: // Execute this Block when condition value evaluates to 0
    break;
  case 0: // Execute this Block when condition value evaluates to 1
          // break is optional
  case 2: // Execute this Block when condition value evaluates to 2
    break;
  default: // Execute this Block when condition no value matches
}
```

- The **break statement** breaks out of the switch statement block once it executes the code associated with the first true case.
- The **default clause** is optional, which specifies the actions to be performed if no case matches the switch condition.

Implement Iterations with Loops

Loops contain the following parts:

- Declaration of the iterator
- Termination condition
- Iterator (typically increment or decrement)
- Loop body

Loop constructs:

- `while` provides a simple iterator.
- `do while` guarantees to get through the loop body at least once.
- `for` keeps declaration, termination condition, and iterator together, on three positions separated with the "`;`" symbol.

```
let i = 0;
while (i < 10) {
  // iterative logic
  i++;
}
```

```
let i = 0;
do {
  // iterative logic
  i++;
} while (i < 10);
```

```
for (let i = 0; i < 10; i++) {
  // iterative logic
}
```

```
while (someFunction()) {
  // iterative logic
}
```

✤ **Note:** `while` and `do/while` loops are best used with a function call that returns Boolean to control iterations.

For Loop Variants

Step through all elements of an array or collection:

- `for of` performs an iteration per element.
- `forEach()` invokes a function per element.

Present array or collection as an argument list:

- `...` (spread operator) flattens all elements.

Step through all object properties:

- `for in` performs an iteration per property.

```
let obj = {a:1, b:2, c:3};
for (const i = 0 in obj) {
  result += obj[i];
}
console.log(result); // 6
```

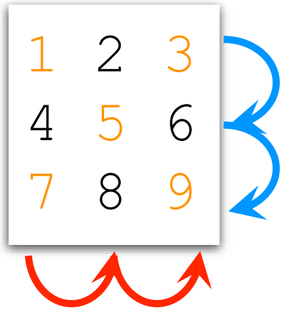
```
let result = 0;
let values = [1,2,3,4,5];
for (const value of values) {
  result += value;
}
console.log(result); // 15
```

```
let result = 0;
let values = [1,2,3,4,5];
values.forEach(function(value) {
  result += value;
});
console.log(result); // 15
```

```
let result = 0;
let values = [1,2,3,4,5];
result = Math.max(...values);
console.log(result); // 5
```

Advanced Loops

- Any type of loop can be nested within any type of loop.
- `continue` skips a loop cycle.
- `break` terminates a loop.
- Blocks of code can be labeled.




```
let result = "";
let data = [[1,2,3],[4,5,6],[7,8,9]];
acme:
for(let x = 0; x < data.length; x++) {
  for(let y = 0; y < data[x].length; y++) {
    if (data[x][y]%2 == 0) {
       continue; // skip even numbers
    }
    result += data[x][y];
    if (x == data.length-1 && y == data[x].length-1) {
       break acme; // skip last "-" symbol
    }
  }
  result += "-";
}
console.log(result); // 13-5-79
```

❖ **Note:** Break and continue statements may use block labels to indicate which loop (inner or outer) they apply to.

Summary

In this lesson, you should have learned to:

- Describe the justification for JavaScript
- Declare and initialize variables
- Use JavaScript data type system
- Perform operations with Text, Numeric, Boolean, and Date values
- Explain value conversions and type casting
- Introduce arrays and collections
- Implement control flow with `if/else`, switch, and loop constructs

Practice Overview

- 2-1: Declare and initialize variables
- 2-2: Create calculations
- 2-3: Process array data
- 2-4: Write conditional logic

ORACLE

3

Functions, Classes, and Objects

Objectives

After completing this lesson, you should be able to:

- Define and use functions, objects, and classes
- Declare and invoke functions
- Create classes and objects
- Manipulate object properties
- Define functions as object properties
- Introduce "`this`" keyword and constructors

Classification of Functions, Classes, and Objects

- A function is a unit of code designed to perform a particular task.
- A class is designed to encapsulate data and functions.
- An object is an instance of a class or a function that represents specific values.

```
function a() { }          // function declaration
let a = function() { }; // function expression (anonymous function)
let aObj = new a();       // object as an instance of a function
a();                      // invoke a function

class b { }               // class declaration
let b = class { } ;       // class expression (anonymous class)
let bObj = new b();       // object as an instance of a class

let cObj = { };           // object can be defined directly
```

❖ Unlike declared functions, anonymous functions and classes are not hoisted.

Define and Invoke a Function

- Function declaration (named) syntax: `function <name> (<parameters>) { <body> }`
- Function expression (anonymous) syntax: `function (<parameters>) { <body> }`
- A function may accept parameters and return a value.

```
function hello1() {
  console.log("Hello World");
}
function hello2(name) {
  console.log("Hello "+ name);
}
function hello3(name) {
  return "Hello "+ name;
}
hello1();
hello2("Joe");
console.log(hello3("Joe"));
```

```
let hello1 = function() {
  console.log("Hello World");
};
let hello2 = function(name) {
  console.log("Hello "+name);
};
let hello3 = function(name) {
  return "Hello "+name;
};
hello1();
hello2("Joe");
console.log(hello3("Joe"));
```


Arrow Functions

An arrow function is a compact variant of an anonymous function expression.

Arrow functions can omit:

- Keyword `function`
- Keyword `return` , when using a single expression
- Round brackets `()` around parameters, when using exactly one argument
- Curly brackets `{}` around function body, when using a single expression

```
let kgToLb = function(kg) {
  return kg*2.20462;
};

let kgToLb = (kg) => {
  return kg*2.20462;
};
let kgToLb = (kg) => kg*2.20462;
let kgToLb = kg => kg*2.20462;

let weight = kgToLb(42);
```

❖ Arrow functions have limitations, because they do not form an object scope.

Global Object

- A global object is provided by the environment in which JavaScript runs.
- It defines functions and properties specific to the environment.
  - In a web browser , a `window` object acts as the global object.
  - Object `window` provides functions such as `alert()`, objects such as `console`, or properties, such as screen dimensions.
- Global variables defined with the `var` keyword belong to the global object.

```
var x = 42;       // this is an equivalent of window.x
let y = 42;       // this is not an equivalent of window.y
console.log(x);   // this is an equivalent of window.console.log(x);
                  // and an equivalent of window.console.log(window.x);
```

✤Top-level variables declared outside any specific function or object are considered global.

`this` keyword

- Keyword **`this`** refers to the current object.
- Inside a function, **`this`** refers to the object that owns this function.
  - Exception: In top-level functions using the `'use strict'` mode, `this` is undefined.

```
var ratio = 2.20462;       // is an equivalent of this.ratio
function a() {
  return this.ratio;       // is 2.20462. is undefined in 'use strict' mode
};
let weight = {
  ratio : 2.20462,
  kgToLb : function() {
    return this.ratio;     // is 2.20462 in any mode
  }
};
```

Define and Access an Object

- Objects can define a comma delimited set of properties.
  - Property definition syntax: `<name>:<value>`
- Property value can be a simple value, a function, or another object.
  - Property access syntax: `<object reference>.<property name>`

```
let e1 = {
  id: 101,
  description: "measure weight",
  measurement: { value:null, unit:null },
  addMeasurement: function(value, unit) {
    this.measurement.value = value;
    this.measurement.unit = unit;
  }
}
```

```
e1.addMeasurement(12,"kg");
// e1.id is 101
// e1.description is "measure weight"
// e1.measurement.value is 12
// e1.measurement.unit is "kg"
```

Converting Objects to String

- Operation `toString()` is available for all JavaScript objects.
- Redefine operation `toString()` to customize conversion to text.

```
let weight = {
  value: 42,
  unit: "kg",
};
weight.toString(); // [object Object]
```

```
let weight = {
  value: 42,
  unit: "kg",
  toString() {
    return this.value+" "+this.unit;
  }
};
weight.toString(); // 42 kg
```


Define and Use Accessors

- Keywords `get` and `set` define accessor functions.
- Accessors are used to add processing to properties.

```
let m1 = {
  value:null,
  unit:null,
  get unitOfMeasure() {
    return this.unit;
  },
  set unitOfMeasure(unit) {
    const units = ["s","m","kg","A","K","mol","cd"];
    this.unit = (units.indexOf(unit) == -1) ? this.unit : unit;
  },
  get measurement() {
    return this.value+" "+this.unit;
  },
  set measurement(value) {
    this.value = (isNaN(value)) ? this.value : value;
  }
};
```

```
m1.measurement = 42;
m1.unitOfMeasure = "kg";
let m = m1.measurement;  // is "42 kg"
```

✤ **Note:** Typically, `get` functions format and `set` functions validate values.

Define and Instantiate a Class

- Class is a template to produce objects.
- Object is an instance of a class created using a `new` operator.
- Constructor implicitly creates properties using the `this.propertyName` syntax.
- Class may define any number of properties and operations, including accessor functions.

```
class Measurement {
  constructor(value, unit) {
    this.value = value;
    this.unit = unit;
  }
  convertKgToLb() {
    return this.value*2.20462;
  }
}
let m1 = new Measurement(42, "kg");
let m2 = new Measurement(12, "m");
m1.value = 12;
m2.unit = "s";
let weight = m1.convertKgToLb();
```

Constructor Functions

- Constructor function behaves similar to a class.
- Function can be used to produce objects using a `new` operator.
- Constructor Function uses the `this.propertyName` syntax to implicitly define object properties.

```
function Measurement(value, unit) {
  this.value = value,
  this.unit = unit,
  this.convertKgToLb = function() {
    return this.value*2.20462;
  }
}
let m1 = new Measurement(42,"kg");
let m2 = new Measurement(24,"m");
m1.convertKgToLb();
```

❖ **Note:** Constructor functions were the closest approximation to the idea of a "class" in JavaScript before actual classes were introduced in EcmaScript ES6 (released in 2015).

Define “Private" Properties

- Properties can still be directly accessed even if accessor functions are defined.
- The property is "private" when its name starts with the # symbol.
- These properties cannot be directly accessed from outside an object.
- Accessing properties without the # symbol results in an attempt to add another "public" property with the same name.

```
class Measurement {
  constructor(value, unit) {
    this.#value = value;
    this.#unit = unit;
  }
  convertKgToLb(){
    return this.#value*2.20462;
  }
}
```

```
let m1 = new Measurement(42,"kg");
m1.convertKgToLb();
```

```
m1.#value = "foo";
m1.#unit = "lb";
```

```
m1.value = "foo";
m1.unit = "lb";
```

Define and Access Static Context

Keyword `static` describes properties and operations of a class shared by all instances.

```
class Measurement {
  #value = null;
  #unit = null;
  static lbToKg(lb) { return lb/2.20462; }
  static units = ["s","m","kg","A","K","mol","cd"];
  set unitOfMeasure(unit) {
    this.#unit = (Measurement.units.indexOf(unit) == -1)
                  ? this.#unit : unit;
  }
  constructor(value, unit) {
    this.#value = value;
    this.#unit = unit;
  }
}
```

```
let lb = 42;
let kg = Measurement.lbToKg(lb);
let m1 = new Measurement(kg, "kg");
let m2 = new Measurement(12, "m");
```


Dynamically Alter an Object

- By default, object structure is not protected or enforced.
- Properties and functions can be added, modified, or deleted on the fly.
- Object value and structure alterations are prevented.

```
let m1 = { unit : "kg", value : 42 };
m1.id = 101;
m1.kgToLb() { return value*2.20462; };
m1.value = 40;
delete m1.unit;
Object.preventExtensions(m1);
Object.seal(m1);
Object.freeze(m1);
```

```
Object.defineProperty(m1, "id", {
  enumerable: true,
  configurable: false,
  writable: false,
  value: 101
});
```

Interrogate Object and Its Properties

- Identify if object has a certain property: `hasOwn()`
- Check if object is extensible (allows addition of new properties): `isExtensible(`*`object`*`)`
- Check if object is sealed (properties are non-configurable): `isSealed(`*`object`*`)`
- Check if object is frozen (no alterations are allowed): `isFrozen(`*`object`*`)`

```
let m1 = {unit:"kg",value:42};
m1.hasOwnProperty("unit"); // true
Object.hasOwn(m1,"unit");  // true
Object.isExtensible(m1);   // true
Object.isSealed(m1);       // false
Object.isFrozen(m1);       // false
```

Retrieve and Populate Object Properties

```
let e1 = {id:101, description: "measure weight"};
let m1 = {unit : "kg", value : 42};

// Extract enumerable properties into a map collection:
let propertyNamesAndValues = Object.entries(m1);
// Extract enumerable property values into an array:
let propertyValues = Object.values(m1);
// Extract enumerable property names into an array:
let propertyNames = Object.keys(m1);
// Extract all property names into an array:
let allPropertyNames = Object.getOwnPropertyNames(m1);

// Populate object with properties from a map collection:
let m2 = Object.fromEntries(propertyNamesAndValues);

// Assign properties from a number of source objects to a given target:
let e2 = Object.assign(e1,m1,m2);
```

Summary

In this lesson, you should have learned to:

- Define and use functions, objects, and classes
- Declare and invoke functions
- Create classes and objects
- Manipulate object properties
- Define functions as object properties
- Introduce "`this`" keyword and constructors

Practice Overview

- 3-1: Wrap application data as object properties
- 3-2: Wrap application logic into functions

ORACLE

4

Extending Classes and Objects

Objectives

After completing this lesson, you should be able to:

- Get an overview of JavaScript extensibility mechanisms
- Extend prototypes
- Extend classes
- Extend objects
- Spread out arrays and objects

JavaScript Extensibility Mechanisms

The purpose of extensibility is to:

- Share common code
- Introduce code variations as required

Extensibility capabilities:

- Add, remove, and modify properties of a given object.
- Add, remove, and modify properties of a number of objects through their prototype.
- Apply prototype of one object to another.
- Use the class inheritance mechanism to extend class *(can be mimicked using a prototype extension mechanism)*.

What Is a Prototype?

- Prototype object is an internal property of any JavaScript object.
- Prototype defines properties and behaviours common for all instances of the object.

```
function x() {}
let x1 = new x();
x.__proto__;                    // function () { [native code] }
x.prototype;                    // Object {constructor: Function}
Object.getPrototypeOf(x);       // function () { [native code] }
x1.__proto__;                   // Object {constructor: Function}
x1.prototype;                   // undefined
Object.getPrototypeOf(x1); // Object {constructor: Function}
```

Create Objects Based on a Specific Prototype

All JavaScript objects inherit properties and methods from a prototype.

- By default, objects are created using the Object.prototype.
- No prototype, or alternative prototype, can be used instead.

```
function f1() {
  this.value=42;
}
function f2() {
  this.value=42;
}
```

```
let m1 = new f1();                  // f1.prototype
let m2 = new f2();                  // f2.prototype
let m3 = Object.create(m1);         // f1.prototype
let m4 = Object.create(null);       // null prototype
f1.prototype.unit = "kg";           // modify f1 prototype
m1.value+" "+m1.unit;               // 42 kg
m2.value+" "+m2.unit;               // 42 undefined
m3.value+" "+m3.unit;               // 42 kg
m4.value+" "+m4.unit;               // undefined undefined
Object.setPrototypeOf(m4,m1);       // f1.prototype
m4.value+" "+m4.unit;               // 42 kg
```

❖ **Note:** Setting prototype after object creation could have significant performance implications.

Function Object

- Every JavaScript function is an instance of the Function object.
- Function object allows to superimpose a function on to another object.
- Invoke a function with a target context object:
  - `function.apply(<target>,<parameter array>)`
  - `function.call(<target>,<parameters>)`
- Create a new function with a target context object:
  - `function.bind(<target>,<parameters>)`

```
let m1 = new Measurement(42,"kg");
let r1 = convert.apply(m1,[2.2]);
let r2 = convert.call(m1,2.2);
let fu = convert.bind(m1,2.2);
let r2 = fu();
```

```
function convert(ratio) {
  return this.value*ratio;
}
class Measurement {
  constructor(value, unit) {
    this.value = value;
    this.unit = unit;
  }
}
```

Extending an Object vs Extending a Prototype

- Adding properties to an object only affects this object.
- Adding properties to a prototype affects all objects based on it.

```
function Lab(location) {
  this.location = location;
}
let lab1 = new Lab("Main");
let lab2 = new Lab("US");
lab2.staff = ["Pinky","Brain"];
Lab.prototype.budget = 123;
```

❖ **Note:** Only `lab2` has `staff` property, but all lab objects have both `location` and `budget`.

Implement Inheritance with Prototypes

- Assign `prototype` property to reference parent object prototype.
- Use `call()` or `apply()` operations to trigger parent object initialization.
- Initialize any other properties.

```
function Lab(location) {
  this.location = location;
}
function FieldLab(location, startDate) {
  //FieldLab.prototype = Lab.prototype;
  Lab.call(this,location);
  this.startDate = startDate;
}
let lab1 = new Lab("Main");
let lab2 = new FieldLab("UK", new Date(2022, 4, 1));
```

❖ **Note:** All instances of `Lab` function contain only `location` property; all instances of `FieldLab` contain both `location` and `startDate` properties.

Implement Inheritance with Classes

- Use `extends` clause to reference a parent class.
- Use `super` keyword to trigger parent object initialization.
- Initialize any other properties.

```
class Lab {
  constructor(location){
    this.location = location;
  }
}
class FieldLab extends Lab {
 constructor(location, startDate){
   super(location);
   this.startDate = startDate;
 }
}
let lab1 = new Lab("Main");
let lab2 = new FieldLab("UK", new Date(2022, 4, 1));
```

Spreading Arrays and Objects

- Spread … Operator.
- Retrieve all elements from the array.
- Retrieve all properties of the object.

```
let arr1 = ['a','b'];
let arr2 = ['c','d'];
// Merge arrays
let arr3 = [...arr1, ...arr2];
// ['a','b','c','d']
```

```
let obj1 = {
    p1:'a',
    p2:'b'
};
let obj2 = {
    p2:'c',
    p3:'d'
};
// Merge objects
let obj3 = {...obj1, ...obj2};
// { p1:'a', p2:'c', p3:'d' }
```

Summary

In this lesson, you should have learned to:

- Get an overview of JavaScript extensibility mechanisms
- Extend prototypes
- Extend classes
- Extend objects
- Spread out arrays and objects

Practice Overview

- 4-1: Extends classes and object
- 4-2: Use object prototypes to extend objects
- 4-3: Dynamically alter object structure
- 4-4: Spread out arrays and objects

ORACLE

5

JavaScript in a Web Browser Environment

Objectives

After completing this lesson, you should be able to:

- Access elements and traverse DOM nodes
- Subscribe to listen to and produce events
- Validate forms
- Extract information from HTML
- Write content to HTML
- Create, update, and remove document nodes

Retrieve DOM Elements

```
<html>
<head>
  <script src="some.js" defer></script>
</head>
<body>
<form action='server url' method='POST'>
  <label for='val'>Value:</label>
  <input id='val' type='text' name='value'>
  <select class='list' id='unit' name='unit'>
    <option value='kg'>Kilogram</option>
    <option value='s'>Second</option>
  </select>
  <input type='submit' value='Add'>
</form>
</body>
</html>
```

```
document.getElementById('val');
document.getElementsByTagName('input');
document.getElementsByClassName('list');
```

Traverse Document Nodes

```
<!DOCTYPE html>
<html>
  <head>
    <script src="some.js" defer>
    </script>
  </head>
  <body>
    ...
  </body>
</html>
```

```
let root = document.getRootNode();
if (root.hasChildNodes()) {
  let nodeList = root.childNodes;
  for (let node of nodeList) {
    switch(node.nodeType) {
      case node.ELEMENT_NODE:
        node.nodeName;
        node.value;
        break;
      case node.ATTRIBUTE_NODE:
      case node.TEXT_NODE:
      // and so on...
    }
  }
}
```

```
let parent = node.parentElement;
```


Interact with Elements and their Attributes

- Acquire a list of attributes names for a given element.
- Retrieve the attribute value using the attribute name.
- Assign a new value to an attribute.
- Create new attributes for an element.
- Explicit conversion of values from string type is strongly advised.

```
<input id='e1' type='text' name='measurement' value='42'>
```

```
let element = document.getElementById("e1");
for (let attName of element.getAttributeNames()) {
  console.log(attName +"="+ element.getAttribute(attName));
}
element.value = "123";
element.disabled = true;
let val = parseInt(element.value);
```

❖ Typical use-case is to acquire form input items to process and validate values.

Interact with Selected Lists

- Acquire a reference to the select element.
- Acquire a list of options.
- Identify which option was selected.

```
<select id='e1' name='unit'>
  <option value='kg'>Kilogram</option>
  <option value='s'>Second</option>
</select>
```

```
let e1 = document.getElementById("e1");
let nodes = e1.childNodes;
for (let node of nodes) {
  if (node.selected) {
    console.log(node.value);
  }
}
```

- Attribute `multiple` is used to allow more then a single item to be selected from a list.

Interact with Check Boxes

- Acquire a reference to the checkbox element.
- Identify whether this check box is checked or not.

```
<input type="checkbox" id="e1" name="complete">
```

```
let e1 = document.getElementById("e1");
let complete = e1.checked;
```


Interact with Radio Buttons

- Acquire a list of radio buttons.
- Identify whether a radio button is checked or not.

```
<fieldset id="unitType">
  <legend>Select Type of Measurement</legend>
  <label for="rb1">Weight</label>
  <input type="radio" id="rb1" name="typeSelector" value="Weight" checked>
  <label for="rb2">Length</label>
  <input type="radio" id="rb2" name="typeSelector" value="Length">
  <label for="rb3">Time</label>
  <input type="radio" id="rb3" name="typeSelector" value="Time">
</fieldset>
```

```
let element = document.getElementById("unitType");
for (let node of element.childNodes) {
  if (node.name == "typeSelector")  {
      console.log(node.value+" "+node.checked);
  }
}
```

Event Handling Overview

Classification of Events

- Window and Document Events, such as a document loading, scrolling, etc.
- UI Element Events, such as mouse actions, focusing on elements, keyboard typing, etc.
- Forms and user input events, such as item changes, form submission, etc.
- HTML Document and Style modifications, such as changes to DOM, animations, etc.

Each event can be associated with a number of handling functions known as event listeners.

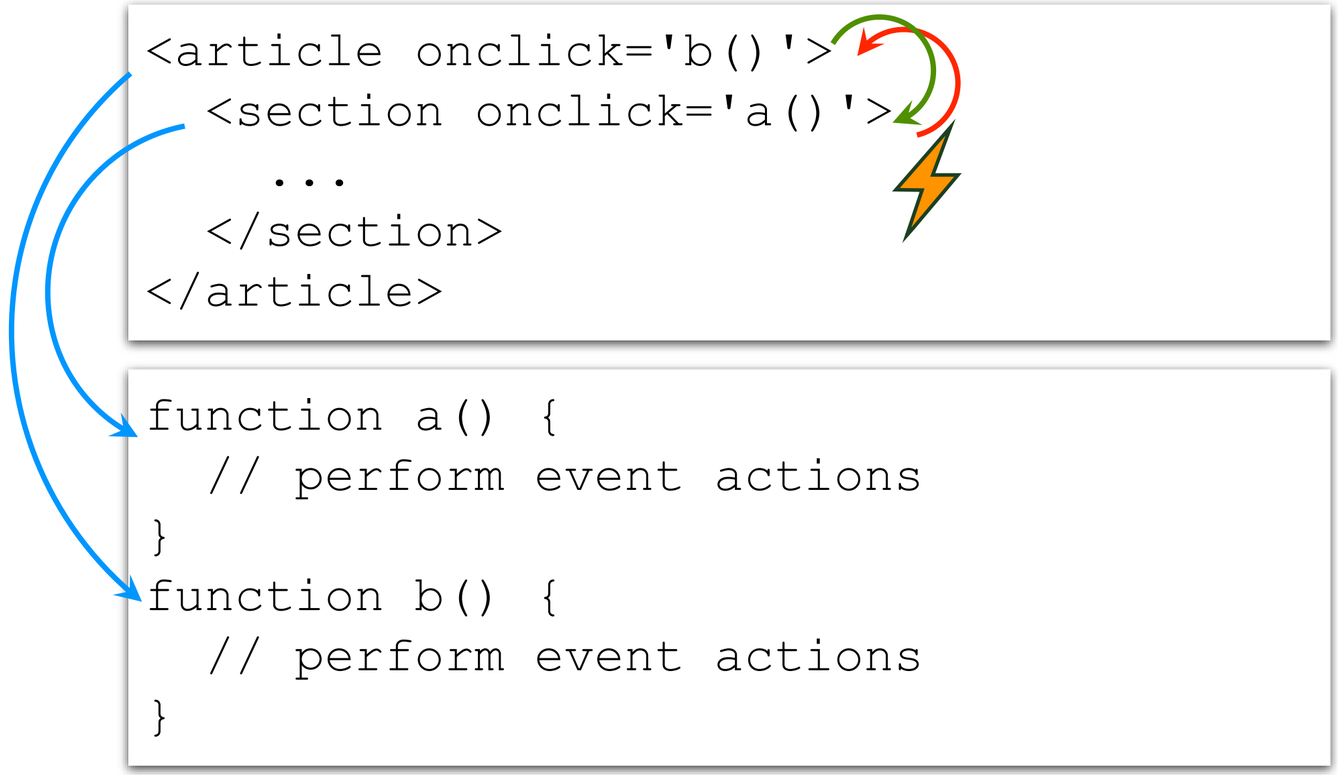
- Event listener can be registered programmatically or declaratively.

```
document.onload = function() {
  let element = document.getElementById('b1');
  element.onclick = myListener;
}
function myListener() { }
```

```
<button id='b1' onclick='myListener()'>
```

Events and Listeners

- Fire event of a certain type.
- Event is "captured" (propagated down) in the element hierarchy.
- Event is "bubbled" (propagated up) in the element hierarchy.
- Matching event listeners are invoked.



```
<article onclick='b()'>
  <section onclick='a()'>
    ...
  </section>
</article>
```

```
function a() {
  // perform event actions
}
function b() {
  // perform event actions
}
```

❖ **Note:** Some events are element specific and thus are not propagated.

Dispatching Events

- Browser automatically fires event in response to user actions or page life cycle phases.
- Both regular and custom events can be fired programmatically.
- An event can be customized using the options object.

```
let e1 = document.getElementById("x");
e1.dispatchEvent(new MouseEvent('click', {
    view: window,
    capture: true,
    bubbles: true,
    cancelable: true
  }));
e1.dispatchEvent(new MyEvent('some', {
   bubbles: true,
   detail: {
     someProperty: someValue
   }}));
```

Subscribing to Events

- An event listener is a function that is invoked when an event is triggered.
- Event listeners can be associated with element via attributes, or programmatically.
- Event listeners can be removed programmatically.
- Multiple functions can listen for the same event and a given element.
- An event parameter provides current event object reference.
- Event object contains context information and controls event propagation.

```
<div id="d1" onclick='a()'>
</div>
```

```
function a() {}
function b(evt) {
  evt.currentTarget;
  evt.stopPropagation();
  evt.stopImmediatePropagation();
}
```

```
document.onload = function() {
  let e1 = document.getElementById("d1");
  e1.addEventListener("click",b);
  e1.removeEventListener("click",a);
}
```

Prevent Default Action

- Event handler can prevent default event action from happening.

```
<form id="frm">
  <input type='text' name='value'>
  <input type='submit' value='Add'>
</form>
```

```
document.onload = () => {
  let frm = document.getElementById("frm");
  frm.addEventListener("submit", formSubmit);
}
formSubmit(evt) {
  evt.preventDefault();
  console.log("Event is triggered, but form is not submitted");
}
```

React to User Input

- Detecting user input
- Detecting field modification
- Referencing an item that has triggered an event

```
<input type='text' name='value' oninput='a(this)'
                                onchange='b(this)'>
<select name='unit' onchange='b(this)'>
  <option value='kg'>Kilogram</option>
  <option value='s'>Second</option>
</select>
```

```
a(item) {
  console.log(item.value);
}
b(item) {
  console.log(item+" is changed");
}
```

❖ **Note:** Alternatively, use the Event object that has a `currentTarget` property to access the element for which the event is triggered.

Validate User Input

In addition to JavaScript functions, UI elements themselves can constrain user input:

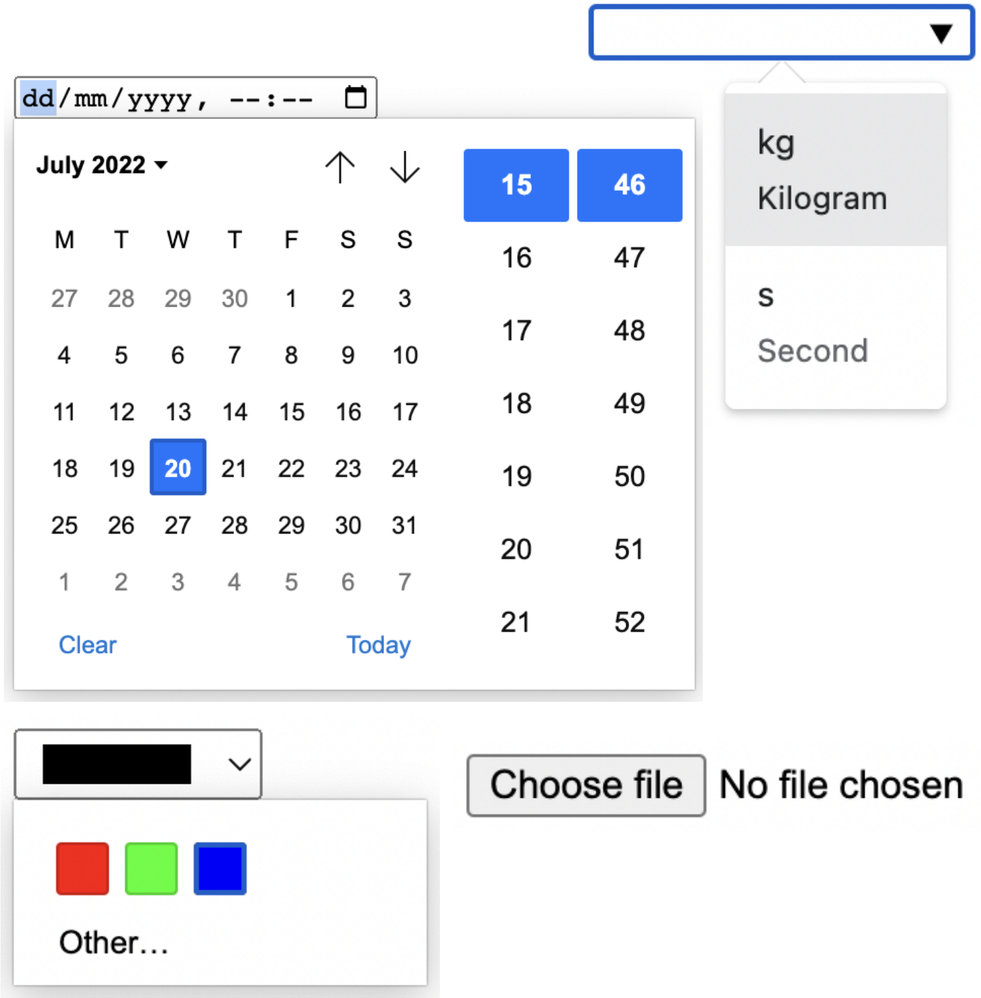
- Type of the input element
- Constraining attributes
- Provides hint of the format of the value

```
<input type='number' name='value'
       required min='-10' max='10'>
<input type='email' name='email'
       placeholder='xxx@yyy.zz'>
<input type='url' name='url'
       placeholder='http://yyy.zz'>
<input type='tel' name='phone'
       pattern='[0-9]{3}-[0-9]{3}-[0-9]{4}'
       placeholder='123-123-1234'>
<textarea name='description'
          minlength='10' maxlength='100'>
</textarea>
```

Assist User Input with Value Selectors

- Provides a list of suggestions for an input element
- Provides the selector for dates, times, colors, files, and so on

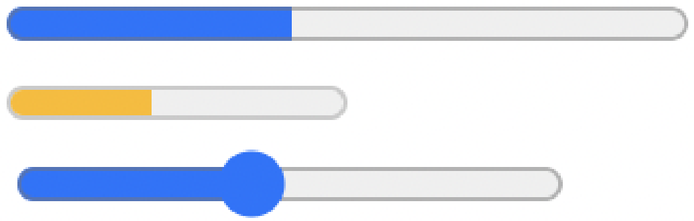
```
<input type="text" list="units">
<datalist id="units">
  <option value='kg'>Kilogram</option>
  <option value='s'>Second</option>
</datalist>
<input type="datetime-local">
<input type="file">
<input type="color" list="rgb">
<datalist id="rgb">
  <option value='#FF0000'>Red</option>
  <option value='#00FF00'>Green</option>
  <option value='#0000FF'>Blue</option>
</datalist>
```



Display and Input a Value within a Range

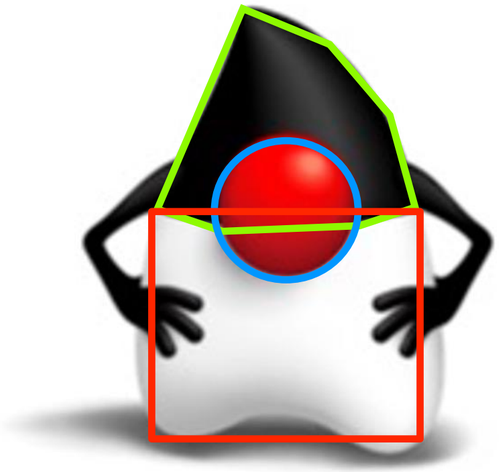
- Both progress bar and meter display a graphic indicator of a value with a range.
- Meter can apply a different style to the value based on which subset of the range it falls into.
- Range input restricts selection of values to that of the range and step.

```
<progress min='0' max='100' value='42'>42%</progress>
<meter min='0' max='100'
       low='33' high='66' optimum='80' value='42'>42%</meter>
<input type='range' min='0' max='100' value='42' step='2'>
```



Produce Image Map

Image Map defines clickable areas for an image.



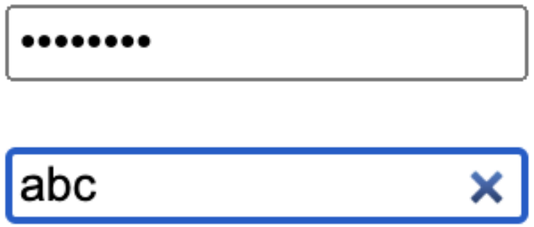
```
<img src="duke.jpg" usemap="#duke-map" alt="Duke">
<map name="duke-map">
  <area alt="nose" title="nose" href="target url"
        coords="113,93,24"
        shape="circle">
  <area alt="head" title="head" href="target url"
        coords="101,23,67,95,89,102,139,97,159,95,142,54,124,35"
        shape="poly">
  <area alt="body" title="body" href="target url"
        coords="67,100,161,174"
        shape="rect">
</map>
```

❖ **Note:** Attribute `alt` is used to provide alternative text if the image is not supported.

Other Input Types

- Password input allows user to securely enter a password, obscuring typed characters.
- Search input is a variant of text input, which usually features a clear button within the field.
- Hidden input is present within the page but does not appear on the screen.

```
<form action="url">
  <input type="password" name="pwd">
  <input type="search" name="search_box">
  <input type="hidden" name="invisible"
         value="not rendered on screen">
</form>
```



Button, Image, and Form Actions

Buttons and clickable images can be used to perform actions, such as:

- Submit form
- Reset form
- Perform any other actions, including submitting and resetting form programmatically

```
<form action="url" id="frm">
  <input type="submit" id="e1" value="Submit">
  <input type="reset" id="e2" value="Reset">
  <input type="button" id="e3" value="Ok">
  <input type="image" id="e4" alt="Duke"  src="duke.jpg">
</form>
```

❖ **Note:** Use a clickable image instead of a button.

Write to HTML Document

- Acquire HTML element reference.
- Write new content, replacing entire content within the target element.

```
<article id='e1'>
  This content is replaced only when setting outerHTML
  <div>This content will be replaced</div>
</article>
```

```
let element = document.getElementById('e1');
let content = '<p>New content</p>';
// element.innerHTML = content;
// element.outerHTML = content;
element.setHTML(content, new Sanitizer());
```

❖ **Note:** Writing to HTML document may have performance and security implications.

Add, Remove, and Modify HTML Document Content

- Construct document fragments comprising one or more nodes.
- Append, insert before, insert after, replace, and clone document nodes.

```
<article id='e1'>
  <div id='e2'>New content will appear before this</div>
</article>
```

```
let parentElement = document.getElementById('e1');
let childElement = document.getElementById('e2');
let newElement = document.createElement('div');
let textNode = document.createTextNode('New content');
newElement.appendChild(textNode);
parentElement.insertBefore(newElement, childElement);
```


Summary

In this lesson, you should have learned to:

- Access elements and traverse DOM nodes
- Subscribe to listen to and produce events
- Validate forms
- Extract information from HTML
- Write content to HTML
- Create, update, and remove document nodes

Practice Overview

- 5-1: Create event listener for load event
- 5-2: Associate event listeners with HTML elements
- 5-3: Read values from HTML elements
- 5-4: Dynamically create or alter HTML elements

ORACLE

6

Cascading Style Sheets

Objectives

After completing this lesson, you should be able to:

- Use CSS Selectors (elements, classes, pseudo-classes, and pseudo-elements)
- Use CSS Properties (sizes, colors, fonts, position, alignment, and orientation)
- Use box model
- Use display layouts such as gird and flex
- Adjust layout based on media capabilities
- Adjust styles programmatically

Defining Styles

- Style target is determined by a selector.
- Style block is delimited with { } and contains a number of properties.
- Style property is a name value pair separated with : and terminated with ; symbol.

```
selector {
  property1:value;
  property2:value;
  /* comments */
}
```

- Elements down the DOM hierarchy inherit style properties from their parents.
- Child elements in the DOM hierarchy may override the inherited style property values.

Element Selectors

- `*` all elements
- `a` all <a> elements
- `.y` all class="y" elements
- `div.y` all elements with class="y"
- `#x` an element with id="x"
- `a, p` all <a> and <p> elements
- `div p` all <p> element descendants of
- `div > p` all <p> children of element
- `a + p` first <p> element after <a>
- `a ~ p` all <p> elements siblings of <a>

```
<div class="y">
  <p> </p>
  <a class="y">
  <p id="x"> </p>
  <p>
    <p> </p>
  </p>
</div>
```

Attribute Selectors

- `[required]` all elements with "required" attribute
- `[value=42]` all elements with value="42"
- `[value~=Meter]` all elements with value attribute containing word "Meter"
- `[name*=u]` all elements with value attribute containing substring "u"
- `[type|=t]` all elements with type attribute equal to or starting with "t"
- `[name^=v]` all elements with name attribute starting with "v"
- `[value$=t)]` all elements with value attribute ending with "m)"

```
<input type="number" name="value" value="42">
<input type="text" name="unit" value="Meter (m)" required>
```

Use Pseudo-Class and Pseudo-Element Selectors

Select document fragments based on their state, behavior, or user actions.

- Elements with certain current state, such as selected, activated, valid or invalid, etc.
- Elements relative to other elements, such as first, last, or nth child elements, etc.
- Fragments of text, such as first line or letter, etc.

Pseudo selector syntax

- Applied to all document fragments that satisfy this selector
- Applied to specific elements that satisfy this selector

```
::pseudo-selector
element:pseudo-selector
```

Not all browsers support all selectors

- Unsupported selectors are ignored.
- Browser-specific selector uses browser type as a name prefix.

```
-moz     // Firefox
-ms      // IE and Edge
-o       // Old Opera
-webkit  // Any Webkit
```

Selecting Elements Based on Their State

- `a:active` an active <a> element
- `a:visited` all visited <a> elements
- `a:link` all unvisited <a> elements
- `#x:target` #x anchor target element when clicked on that anchor
- `a:hover` an <a> element when a cursor moves over it
- `input:focus` an <input> element that has focus
- `::selection` selected portion of a document

Selecting Elements Based on Their Properties

- `input:default` an default <input> element in each group
- `input:disabled` all disabled <input> elements
- `input:enabled` all enabled <input> elements
- `input:in-range` all <input> elements with a value within a range
- `input:out-of-range` all <input> elements with a value outside a range
- `input:read-only` all read-only <input> elements
- `input:read-write` all non-read-only <input> elements
- `input:required` all required <input> elements
- `input:optional` all non-required <input> elements
- `input::placeholder` all <input> elements with placeholder specified
- `input:valid` all valid <input> elements
- `input:invalid` all invalid <input> elements

Selecting Child Elements within a Parent

Selecting child elements:

- `p:first-child` a <p> element if it is the first in a group of siblings
- `p:nth-child(even)` every even <p> element in a group of siblings
- `p:nth-last-child(2)` a <p> element if it is the second from the end of a group
- `p:only-child` a <p> element when it's the only child of its parent

Selecting child elements (among siblings of the same type):

- `p:first-of-type` first <p> element within a group of siblings
- `p:nth-of-type(odd)` every odd <p> element in a group of siblings
- `p:nth-last-of-type(2)` second from the end <p> element in a group of siblings
- `p:only-of-type` a <p> element when it's the only <p> child of its parent

Other Selectors

Selecting fragments of text:

- `p::first-letter` first letter of every <p> element
- `p::first-line` first line of every <p> element

Some other notable selectors:

- `li::marker` bullet marker for all <li> elements
- `p:lang(fr)` all <p> elements with attribute lang="fr" (French)
- `:not(p)` all elements that are not <p>
- `:root` the document root

Visual Properties

Property categories:

- Size
- Color
- Font
- Position, alignment, and orientation
- And so on

```
selector {
  property1:value;
  property2:value;
}
```

Properties customize:

- Object appearance and behaviour
- Animation, speech, audio, video, and subtitles
- And so on

❖ Note: Specific properties depend on the capabilities of a target element.

Setting Sizes

CSS sizes can use different units and define font size, object size, distance between objects, etc.

- Absolute sizing units:
  - `cm  centimeter  1cm`
  - `mm  millimeter  1mm = 0.1cm`
  - `in  inch.  1in = 2.54cm`
  - `px  pixel  1px = 0.026cm 1/96 of an inch (commonly used)`

  (actual number of real pixels is relative to a screen resolution)
  - `pt  point  1pt = 0.35cm  1/72 of an inch`
  - `pc  picas  1pc = 0.42cm  1/6  of an inch`

- Relative sizing units
  - `%  relative to parent element (commonly used)`
  - `em  relative to font size (commonly used)`

Setting Colors

Colors can be set using:

- Color names

```
red; darkgreen;
```

- Red, Green, Blue + Alpha
  - Hexadecimal notation, range between 00 and FF per each channel (commonly used)

```
#FFFF00;   // yellow
#00FF007F; // green, 50% opacity
```

  - Decimal notation, range between 0 and 255 per each channel

```
rgb(255,0,255);      // fuchsia
rgba(0,0,255,0.5);  // blue, 50% opacity
```

- Hue, Saturation, Lightness + Alpha

```
hsl(120, 100%, 25%);         // dark green
hsla(120, 100%, 25%, 0.5); // dark green, 50% opacity
```

- Gradient

```
linear-gradient(270deg, red, orange, yellow);
```

Selecting Fonts

Font family approximates font selection to a group of fonts.

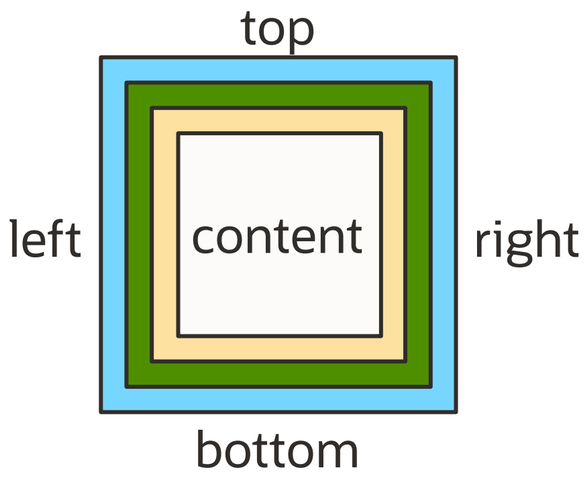
- serif: Times, Times New Roman
- sans-serif: Helvetica, Arial
- monospace: Courier, Courier New
- cursive: Apple Chancery, Comic Sans MS
- fantasy: Papyrus, ALGERIAN

```
font-family: "Lucida Console", "Courier New", monospace;
font-style: italic;
font-weight: bold;
font-size: 20px;
text-decoration: underline overline;
text-decoration-color: orange;
text-decoration-style: dotted;
text-align: center;
```

Positioning, Sizing and Orientation using Box Model

Box areas

- Padding – space between box content and its border
- Border – visible or invisible box boundary
- Margin – space between the box and other objects



Box edges

- Top, bottom, left, right

Content alignments

- One of the edges, middle, or center
- Component orientation within the box

Box Model Example


```
<div class='box1'>
  Duke says:
  <div class="box2">"Hello!"</div>
  <div class="box2">
    <img class="img1" src="duke.jpg">
  </div>
  from a centred box.
</div>
```

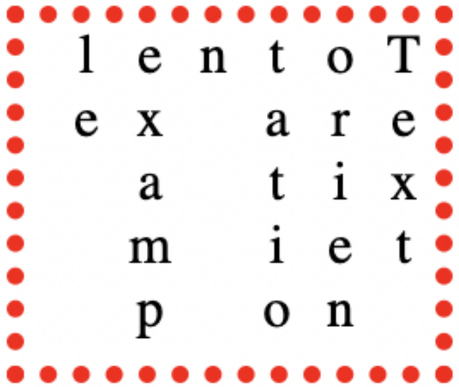
```
.box1 {
    background-color: lightyellow;
    width: 200px;
    border: 15px dashed orange;
    padding: 30px;
    margin: 10px;
    text-align: center;
}
.box2 {
    margin: 10px;
    color: orange;
    text-align: center;
}
.img1 {
  border-style: groove;
  border-width: 5px;
  max-width: 150px;
  max-height: 150px;
}
```

Component Orientation

Component can be oriented in different directions:

- Vertical or horizontal
- Left-to-right or right-to-left

Default direction depends on the language.




```
<div class="box">
  Text orientation example
</div>
```

```
.box {
  border-width: 5px;
  border-style: dotted;
  border-color: red;
  height: 100px;
  width: 120px;
  word-wrap: anywhere;
  writing-mode: vertical-rl;
  text-orientation: upright;
}
```

Handling Overflow

Overflow occurs when content does not fit its container dimensions.

- Wrap content
- Use scroll behavior


```
<div class="box">
  Text orientation example
</div>
```

```
.box {
  border-width: 5px;
  border-style: dotted;
  border-color: red;
  height: 40px;
  width: 70px;
  scroll-behavior: smooth;
  overflow: scroll;
  word-wrap: anywhere;
}
```

Control Display Layout

The display property controls the arrangement of content within a given container.

- Content can be arranged as `grid`, `flex`, `table`, `list`, `block`, and so on.
- Arrangement properties affect positioning, alignment, gaps, padding, margins, and behaviors of objects within this layout.

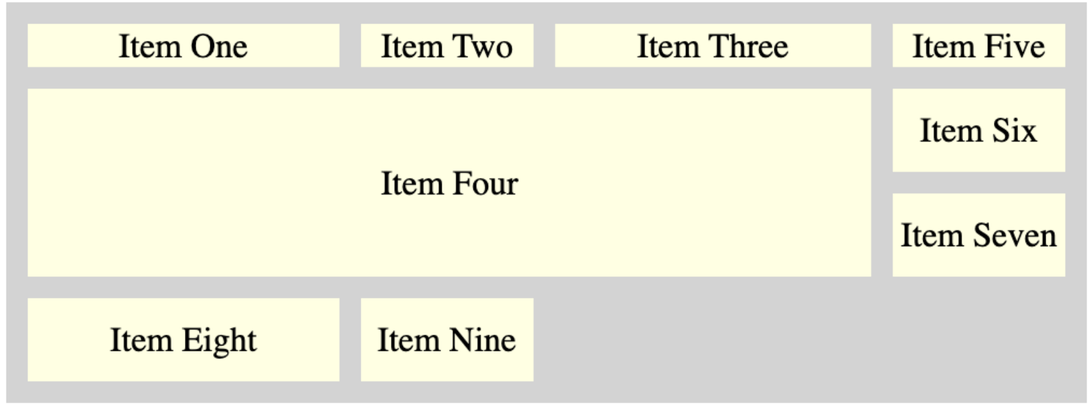
```
.container {
  display: /* type */;
  /* arrangement properties */
}
```

```
<div class="container">
  <div>Item One</div>
  <div>Item Two</div>
  <div>Item Three</div>
  <div>Item Four</div>
</div>
```

Grid Display

Describes columns, rows, areas, gaps, and sizing of the grid

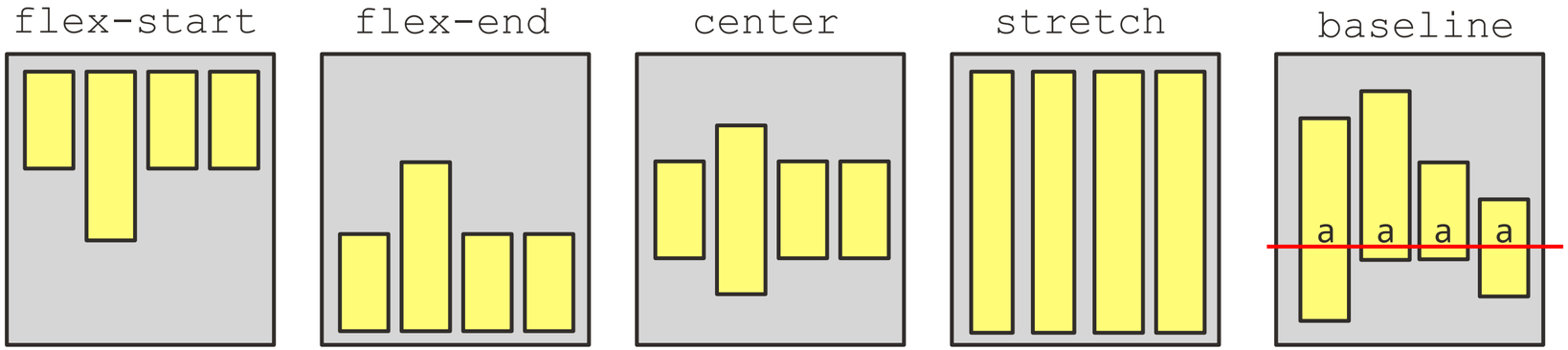
```
.grid {
  display: grid;
  grid-template-columns: auto 80px auto 80px;
  grid-template-rows: 20px auto auto auto;
  grid-gap: 10px;
  background-color: lightgrey;
  padding: 10px;
}
.area {
  grid-area: 2 / 1 / span 2 / span 3;
}
.box {
  display: flex;
  background-color: lightyellow;
  justify-content: center;
  align-items: center;
  padding: 10px 0;
}
```



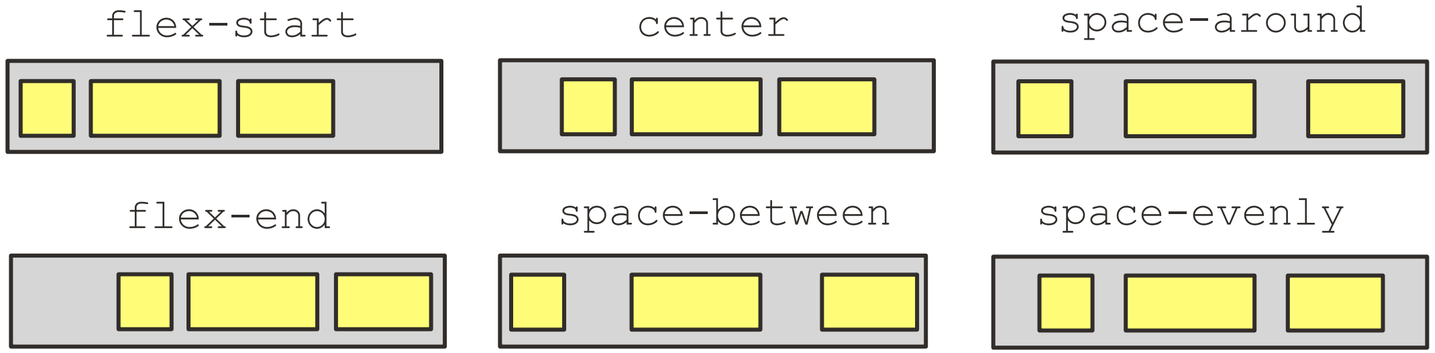

```
<div class="grid">
  <div class="box">Item One</div>
  <div class="box">Item Two</div>
  <div class="box">Item Three</div>
  <div class="box area">Item Four</div>
  <div class="box">Item Five</div>
  <div class="box">Item Six</div>
  <div class="box">Item Seven</div>
  <div class="box">Item Eight</div>
  <div class="box">Item Nine</div>
</div>
```

Flex Flow and Alignments

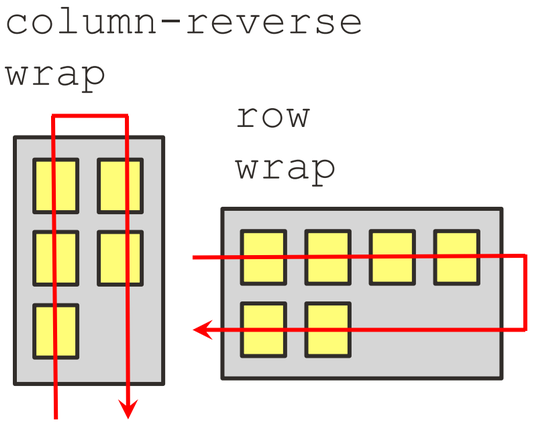
- Property `align-items` controls cross axis alignment



- Property `justify-content` controls main axis alignment



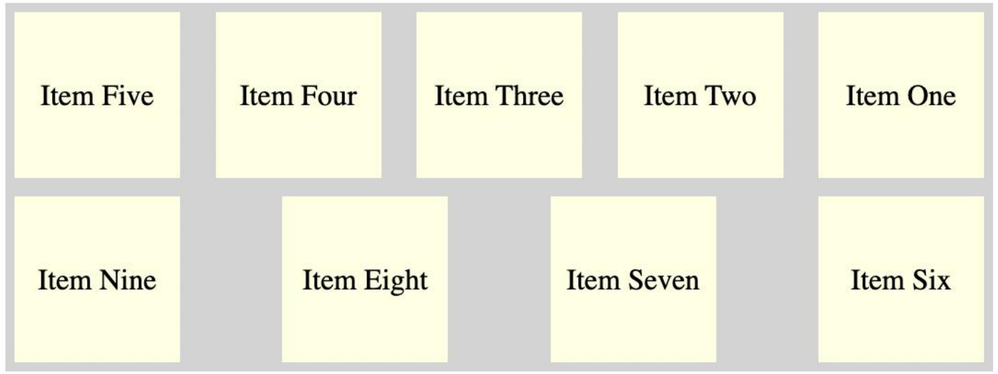
- Property `flex-flow` controls direction and wrapping



Flex Display

Describes a flow of components with spacings and alignments

```
.container {
  display: flex;
  background-color: lightgrey;
  height: 200px;
  flex-flow: row-reverse wrap;
  align-content: stretch;
  justify-content: space-between;
}
.box {
  display: flex;
  background-color: lightyellow;
  justify-content: center;
  align-items: center;
  text-align: center;
  width: 80px;
  padding: 5px;
  margin: 5px;
}
```

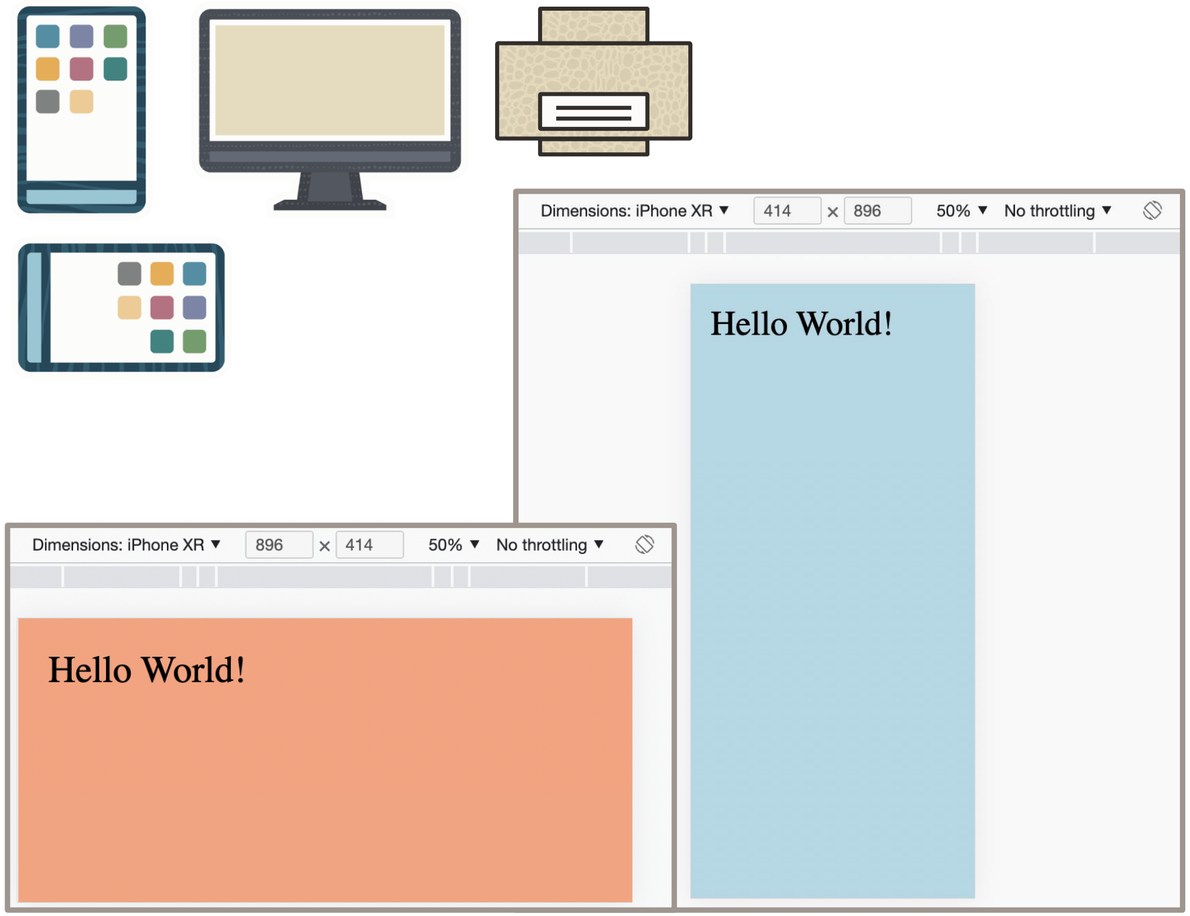



```
<div class="container">
  <div class="box">Item One</div>
  <div class="box">Item Two</div>
  <div class="box">Item Three</div>
  <div class="box">Item Four</div>
  <div class="box">Item Five</div>
  <div class="box">Item Six</div>
  <div class="box">Item Seven</div>
  <div class="box">Item Eight</div>
  <div class="box">Item Nine</div>
</div>
```

Altering Styles Based on Required Media

Select a different style based on the client's media capability.

```
@media screen
       and (min-device-width: 300px)
       and (max-device-width: 800px){
  body {
    background: lightblue;
    font-size: 120px;
    padding: 60px;
  }
}
@media screen
       and (min-device-width: 801px) {
  body {
    background: lightsalmon;
    font-size: 60px;
    padding: 30px;
  }
}
```



Media Queries

Identify client media capabilities

- Mode: `screen`, `print` preview, `all` (either screen or preview)
- Dimensions such as min, max, ratio, orientation
- Color pallet capabilities
- Boolean logic: `and or not only`

```
@media screen
    and (min-width: 900px)
    or  (max-height: 1200px)
    not (display-mode: browser)
    and (max-aspect-ratio: 16/9)
    and (orientation: landscape) {
    /* style to be applied when all media query conditions are true */
}
@media only print and (monochrome) {
    /* style to be applied when all media query conditions are true */
}
```

Dynamic Style Changes

Switch class attribute value for an HTML element using `classList` property.

Operations: `add()` `toggle()` `replace()` `contains()` `remove()`

```
window.onload = function() {
  let e1 = document.getElementById("e1");
  e1.addEventListener("click",switchLayout);
};

function switchLayout(evt){
  let item = evt.currentTarget;
  if (item.classList.contains("styleA")) {
    item.classList.replace("styleA", "styleB");
  }else{
    item.classList.replace("styleB","styleA");
  }
}
```

Summary

In this lesson, you should have learned how to:

- Use CSS Selectors (elements, classes, pseudo-classes, and pseudo-elements)
- Use CSS Properties (sizes, colors, fonts, position, alignment, and orientation)
- Use box model
- Use display layouts such as gird and flex
- Adjust layout based on media capabilities
- Adjust styles programmatically

Practice Overview

- 6-1: Create CSS document
- 6-2: Apply styles to page elements
- 6-3: Use dynamic styles
- 6-4: Scale display based on media query

ORACLE

7

Advanced JavaScript

Objectives

After completing this lesson, you should be able to:

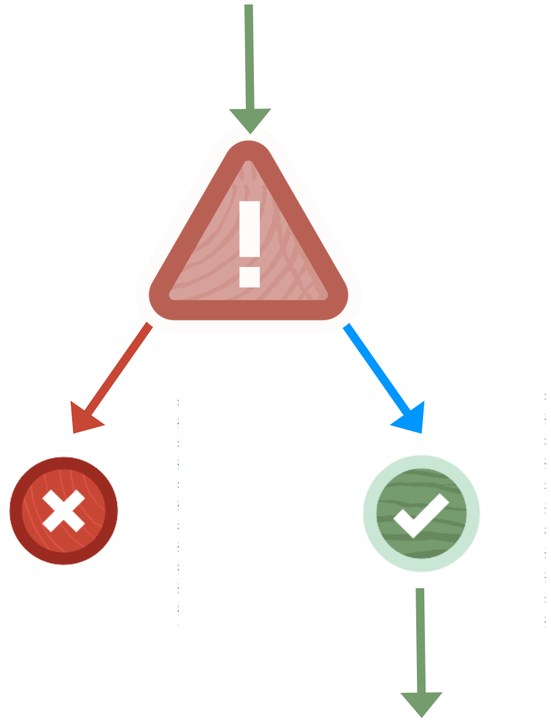
- Handle exceptions with try-catch-finally construct
- Use promises
- Use async await construct
- Use closures
- Create and use modules
- Implement concurrency

Exception Handling

Exception is an unexpected event that occurs within a program.

When exception occurs:

- Normal execution flow is interrupted
- Error handling flow is triggered
- Normal flow can resume if exception was successfully handled
- Unhandled exception terminates program flow



Producing Exceptions

Exception can be produced by:

- JavaScript run time *(for example, an attempt to invoke nonexistent function)*
- Custom program logic
  - Keyword `throw` is followed by `error expression.`
  - Error expression can be any object or primitive value.

When no error handling is present, the remaining program flow is not executed.

```
abc(); // non-existent function
// remaining logic
```

```
divide(a, b) {
  if (b == 0) {
    throw "Cannot divide by zero";
  }
  return a/b;
}
```

Error expression examples:

```
throw "some message";
throw 123;
throw true;
throw new Error("some message");
```

Handling Exceptions

- A number of statements that can produce exceptions are placed inside the `try` block.
- `try` block must be followed by a `catch` and/or `finally` block, or both.
- Exceptions interrupt the `try` block execution flow.
- Flow control is transferred to the `catch` block if it exists.
- Normal flow only resumes upon successful completion of the `catch` and/or `finally` blocks.
- `finally` block is always executed regardless whether exception did or did not occur.

```
try {
  /* actions that can produce exceptions */
} catch (exception) {
  /* exception handling actions */
} finally {
  /* actions that are always executed */
}
/* the rest of the program */
```

Timeouts and Intervals

A function can be executed:

- With a specified delay in milliseconds
- Periodically with a specified interval in milliseconds

Both timeout and interval actions can be canceled.

```
let i = setInterval(showTime, 1000, "en", "US");
let t = setTimeout(showTime, 1000, "en", "GB");
clearInterval(i);
clearTimeout(t);
showTime(language, country){
 console.log(new Date().toLocaleString(language+"-"+country));
}
```

❖ **Note:** Exact timeout of interval is not guaranteed, but it will be at least amount specified.

Promises

A promise object represents a function with two possible outcomes:

- A successful completion of a promise
- A rejection of a promise

Reacting to the promise outcome:

- One or more `then` actions that are triggered upon promise successful resolution
- A `catch` action that is triggered upon promise rejection
- A `finally` action that is always triggered

```
let choice = experimentComplete();
let p = new Promise((res, rej) => {
if (choice) {
  res('resolved');
} else {
  rej('rejected');
}});
```

```
p.then((message) => {
  console.log("Promise "+message);
}).catch((message) => {
  console.log("Promise "+message);
});
```

Execute an Array of Promises

- `all` presents resolved promises as an array or triggers catch if any promise was rejected.
- `allSetteled` presents all resolved and rejected outcomes as a single array.
- `any` completes upon the first resolved promise or triggers catch if all promises were rejected.
- `race` completes upon the first resolved or rejected promise.

```
let p1 = new Promise((res, rej)=>{ ... });
let p2 = new Promise((res, rej)=>{ ... });
let p3 = new Promise((res, rej)=>{ ... });
Promise.all([p1,p2,p3])
       .then((results)=>{ ... })
       .catch((result)=>{ ... });
Promise.race([p1,p2,p3])
       .then((result)=>{ ... })
       .catch((result)=>{ ... });
```

Async/Await

Keyword `async` makes function implicitly wrap returned value into a promise object.

Keyword `await` is:

- Only allowed inside async functions
- Pauses and waits for the promise to resolve
- Can be applied to any "promise compatible" object if it defines then() function

```
async function doThings() {
  return "whatever";
}

doThings().then(...)
          .catch(...)
          .finally(...);
```

```
async function doThings() {
  let promise = new Promise((res, rej) => {
    setTimeout(() => res("ok"), 1000)
  });
  await promise;
}
```

Pass Functions as Arguments and Return Functions

Function can be passed as an argument to another function.

- Reuse function logic in a different context.

Function can be returned from a function (Closure).

- "Preconfigure" *(provide a separate enclosed scope)* for a function before it is returned.
- It is a way of producing immutable objects and emulating private methods.

```
function write(message) {
  console.log(message);
}
function doThings(writer) {
  writer("Hi!");
}
doThings(write);
```

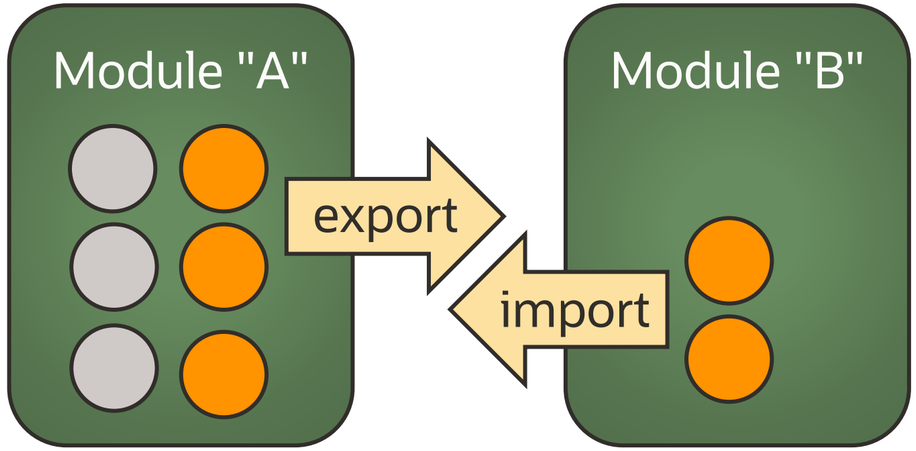
```
function doThings(message) {
  return function write() {
    console.log(message);
  };
}
let writer = doThings("Hi!");
writer();
```

Modules

A mechanism to group JavaScript components such as classes, functions, and variables

Each JavaScript Module:

- Is produced as a separate script file
- Describes (exports) which of its components should be visible to other modules
- May reference (imports) some of the components exposed by other modules



Export Components

Only exported components can be imported and used by other modules.

- Export any classes, functions, and variables.
- Optionally define the default export of a specific component.
- Export components immediately as they are defined.
- Export as separate statements.

duke.js

```
class experiment { }
const measurements = [ ];
function convert() { }
export default experiment;
export {measurement, convert};
```

```
export default class experiment { }
export const measurements = [ ];
export function convert() { }
```


Import Components

Import allows to use components defined in another module.

- It refers to another module file.
- It wraps any number of components within the import object.
- The default exported component can be imported without wrapping.
- Optionally, imported components can be aliased.

duke.js

```
export default experiment;
export {measurements, convert};
```

main.js

```
import exp, {measurements as mes,
             convert} from './duke.js';
let e1 = new exp(); // use alias to create an instance of experiment
```

Access Modules from HTML Document

- Page only needs to reference primary modules that import other modules.
- `script type` must be set as `module`.
- For older browsers that do not support modules, an alternative script may be provided.

duke.js

```
export {experiment, measurements};
```

main.js

```
import {experiment, measurements} from './duke.js';
```

page.html

```
<script type='module' src='main.js'></script>
<script type='nomodule' src='old.js'></script>
```

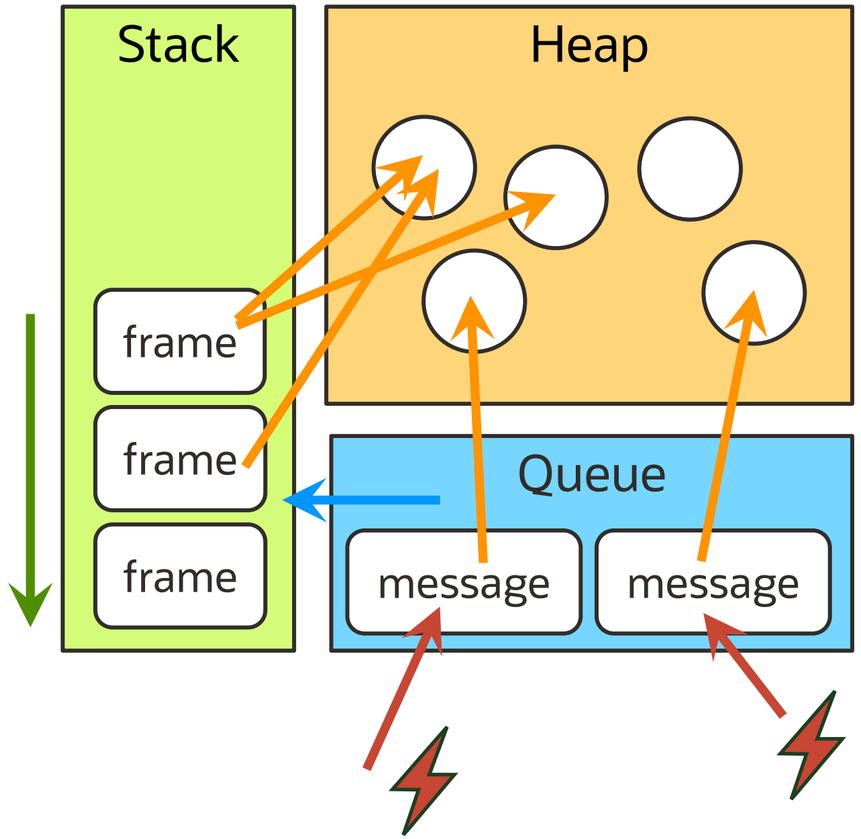
JavaScript Memory Organization and Code Execution

- All objects are placed in a Heap.
  (Other memory areas refer to heap objects as required.)

- Each function invocation creates its own Frame.
  (a context in which function is executed)
- Frames are executed within their order in the Stack.

- Messages are concurrently created requests to handle events or invoke functions.
- Each message is associated with a function call and its context, such as parameters or outer function variables.

- Event Loop dequeues messages from the queue and processes each message through the stack.
- Messages and Frames are processed sequentially.



JavaScript Concurrency Limitations

JavaScript has very limited concurrent processing capabilities.

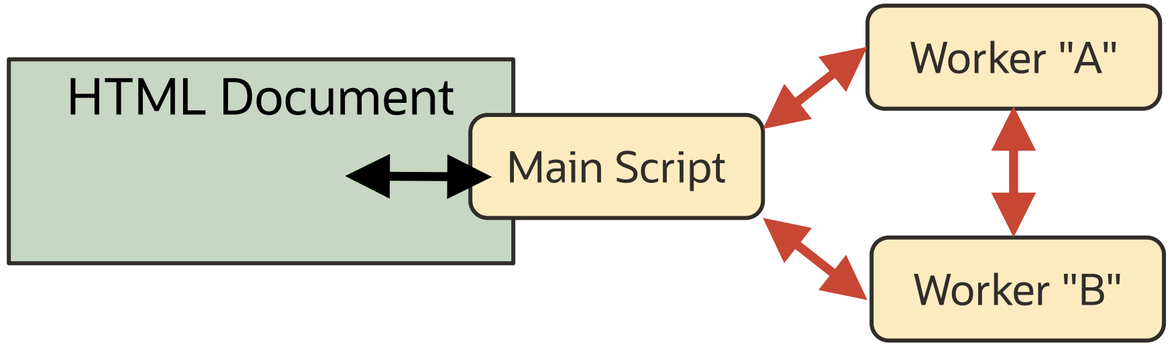
- JavaScript program runs in a single-threaded event loop.
- Concurrent events and requests to execute functions are processed sequentially through the message queue.
- `setTimeout` and `setInterval` functions can mimic some of the concurrency aspects but are actually executed sequentially through the same message queue.

Execute Concurrent Code with Workers

Worker API allows to execute concurrent code in the background.

Worker capabilities and limitations:

- Runs in separate threads using own message queue per worker
- Cannot directly interact with user interface or access DOM
- Exchanges information between each other and a main program via messages
- Gets high start-up performance and memory penalties
- Is useful for long-running, processing intensive background tasks



Worker Life Cycle

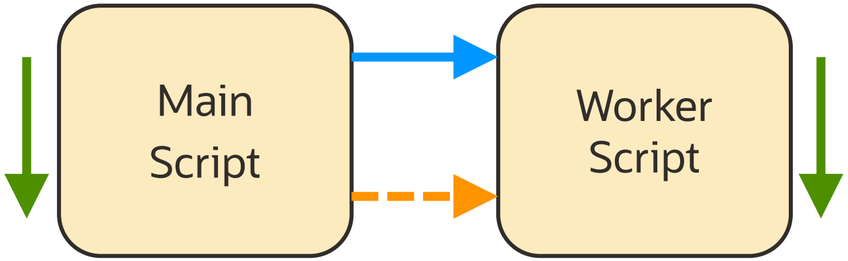
- Each Worker is implemented in its own script file.
- Worker starts execution immediately when it is instantiated.
- Worker is stopped when its work is done, or by using the `terminate` function.

main.js

```
let w = new Worker('./task.js');
w.terminate();
```

task.js

```
/*
Any processing that needs to be
performed as a background thread
*/
```



Worker Message Exchanges

Operation `postMessage` sends messages between workers and the main program.

- It accepts a `message` parameter and an optional `transfer` parameter.
- A `message` event is used to receive and process posted messages.
- It assigns event listener to `onmessage` property.
- It registers `message` event listener.

main.js

```
let w = new Worker('task.js');
let obj = {u:'kg',v:42};
w.postMessage(obj);
w.onmessage = (msg) => {
  console.log(msg.data);
};
```

task.js

```
this.addEventListener("message", (msg) => {
  console.log(msg.data);
  this.postMessage("Confirmed");
});
```

❖ **Note:** Message event listener can post messages back to the program that has triggered the event.

Summary

In this lesson, you should have learned to:

- Handle exceptions with try-catch-finally construct
- Use promises
- Use async await construct
- Use closures
- Create and use modules
- Implement concurrency

Practice Overview

- 7-1: Use promises
- 7-2: Structure code using async-await construct
- 7-3: Define and use a module

ORACLE

8

Exchanging Data with Servers

Objectives

- Produce and parse JSON
- Invoke REST Services
  - Use the old-style AJAX client
  - Use modern style promises and the async/await client
- Implement WebSockets interactions
- Manage application state with parameters, headers, cookies, and local and session storage

What is JSON?

JSON (JavaScript Object Notation)

- Lightweight, text-based, language-independent data exchange format
- JSON can represent two structured types:
  - Object, an unordered collection of zero or more names: value pairs, enclosed in { }
  - Array, an ordered sequence of zero or more values, enclosed in [ ]
- Values can be strings, numbers, booleans, null, objects, and arrays.
- The rest of the object properties, such as functions, are omitted from JSON.

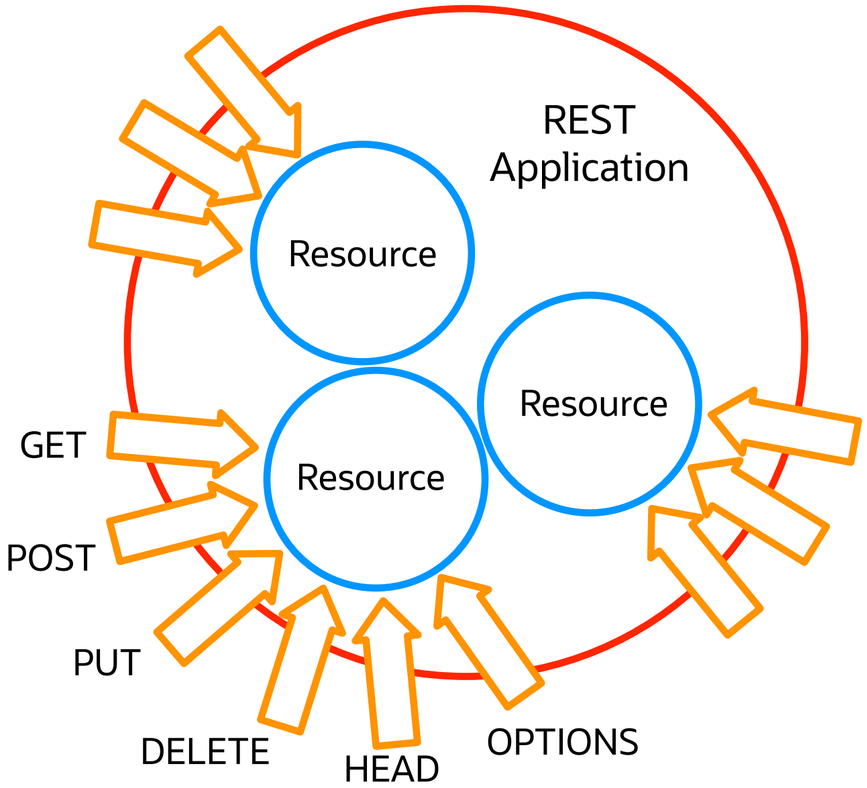
```
{"task":"Measure temperature change",
 "measurements":[
    {"value":291.25, "unit":"K"},
    {"value":289.55, "unit":"K"}
]}
```

```
let obj = JSON.parse(json);
let json = JSON.stringify(obj);
```

Parsing and formatting JSON

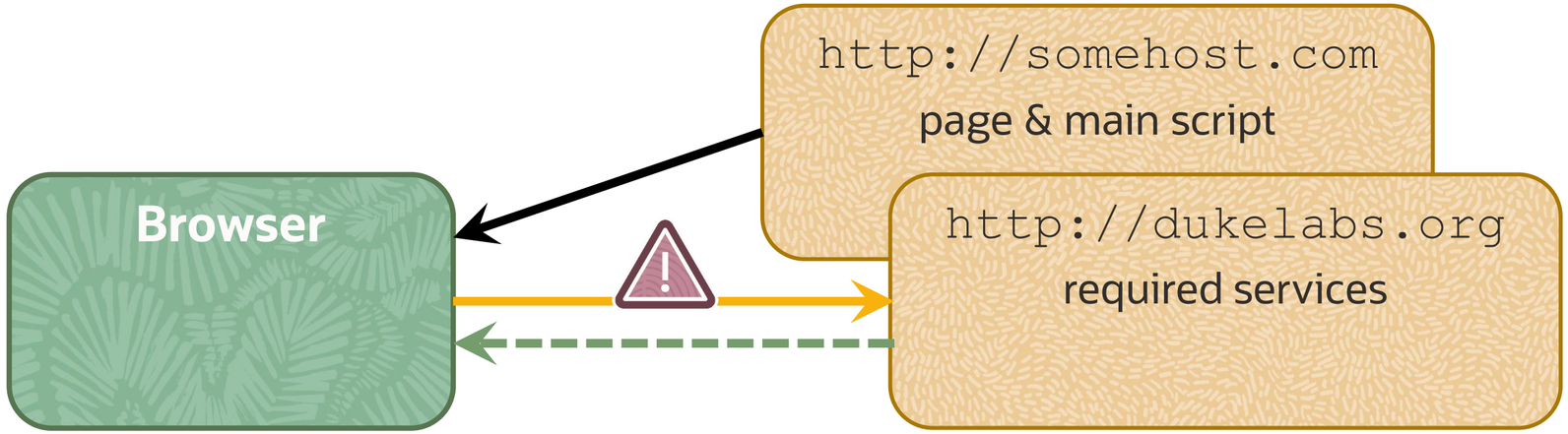
REST Service Conventions and Resources

- A REST service application utilizes one or more Resources.
- Each Resource represents a business entity and is mapped to its own URI.
- Each Resource defines a number of operations mapped to HTTP Methods.
- Typical conventional use of HTTP Methods:
  - GET receives (queries) a collection of objects or a specific object identified by a client.
  - POST creates a new object.
  - PUT updates an existing object.
  - PATCH updates a portion of an object.
  - DELETE removes an object.
- Other operations are sometimes used to retrieve metadata.
- REST messages could be in any format (for example, JSON).



Cross-Origin Resource Sharing (CORS)

- By default, browser-based JavaScript programs are restricted from interacting with remote services that are not located on the same server as the page where the JavaScript program runs.
- Cross-Origin Resource Sharing (CORS) allows the server to use HTTP headers to describe the origin of the resource other than its real origin.
- Browsers can precheck the information about the origin of the script before allowing it to run.




```
Access-Control-Allow-Origin: http://dukelabs.org
Access-Control-Allow-Credentials : true
Access-Control-Allow-Methods: POST. GET. OPTIONS
Access-Control-Allow-Headers: Origin, X-Requested-With
```

Dispatch Service Request Using AJAX

Asynchronous JavaScript and XML API (AJAX):

- Supports both synchronous and asynchronous interaction styles
- Handles different types, such as plain text, XML, or JSON
- Prepares HTTP requests using methods such as GET, POST, PUT, and DELETE
- Dispatches HTTP requests, optionally uploading the request body object as an argument
- Registers a callback function to handle the response from the service

```
let request = new XMLHttpRequest();
request.open("PUT","http://dukelabs.org/experiment/101");
request.send(jsonValue);
request.onload = () => {
  // handle response
};
```

Handle Service Response Using AJAX

- Identify the response status using HTTP Status Codes.
- Extract the response message body.
- Convert response values to a representation preferred by the client.
- Raise exceptions or otherwise signal to the program if the response is erroneous.

```
request.onload = () => {
  if (request.status == 200) {
    let obj = JSON.parse(request.response);
    // use values received from the service
  } else {
    throw new Error("Error "+request.status+" "+request.statusText);
  }
}
```

Dispatch Service Requests Using Modern JavaScript

- Use the `fetch` operation to dispatch the request.
- The operator `await` returns the service response represented as a `Promise` object.

```
async function updateExperiment(experiment) {
  let response = await fetch("http://dukelabs.org/experiment/101", {
    method: 'PUT',
    headers: {'Content-Type': 'application/json'},
    body: JSON.stringify(experiment)
  });
  /* handle response */
}
```


Handle Service Response Using Modern JavaScript

- Identify the response status using HTTP Status Codes.
- Use the `await` operator to extract the response message body represented as a promise.
- Convert the JSON service response to a JavaScript object.
- Raise exceptions or otherwise signal to the program if the response is erroneous.

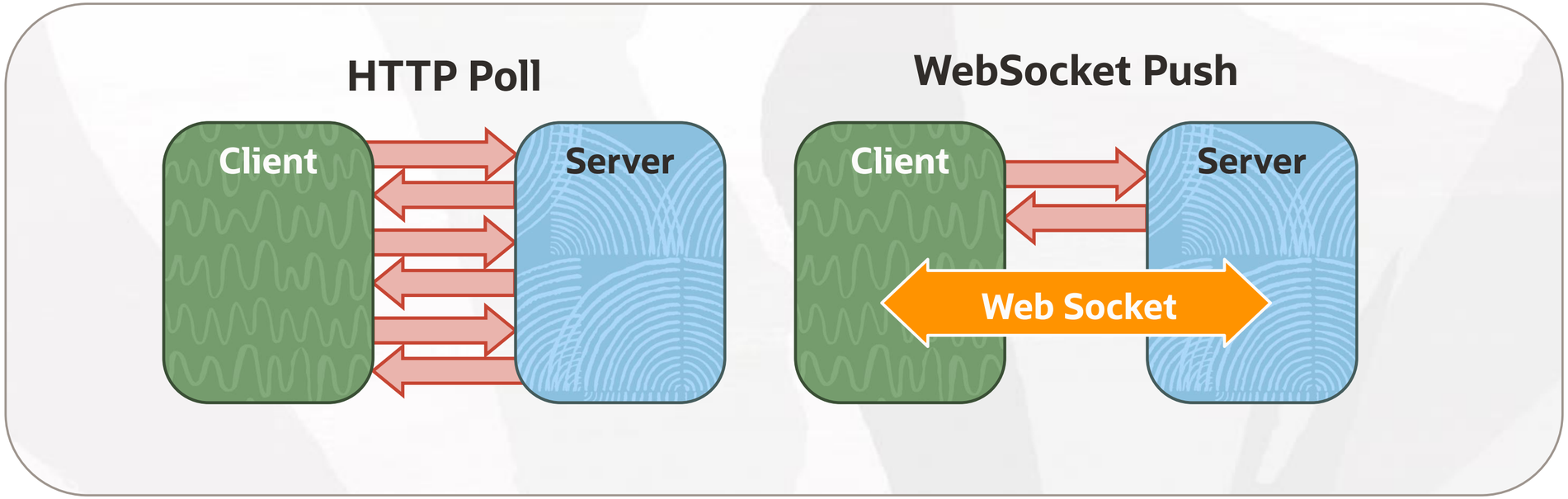
```
async function updateExperiment(experiment) {
  /* initiate request and acquire response */
  if (response.status == 200) {
    experiment = await response.json();
    return experiment;
  }
  throw new Error("Error "+response.status+" "+response.statusText);
}

updateExperiment(experiment).then(data => {console.log(data);})
                            .catch(error) => {console.log(error);};
```

Introduction to WebSockets

WebSockets is a network protocol that:

- Enables stateful, bidirectional communication between client and server
- Allows either side (client or server) to send or receive messages in any order
- Allows the server to push information to the client
- Starts a connection with an HTTP handshake but does not use HTTP after that

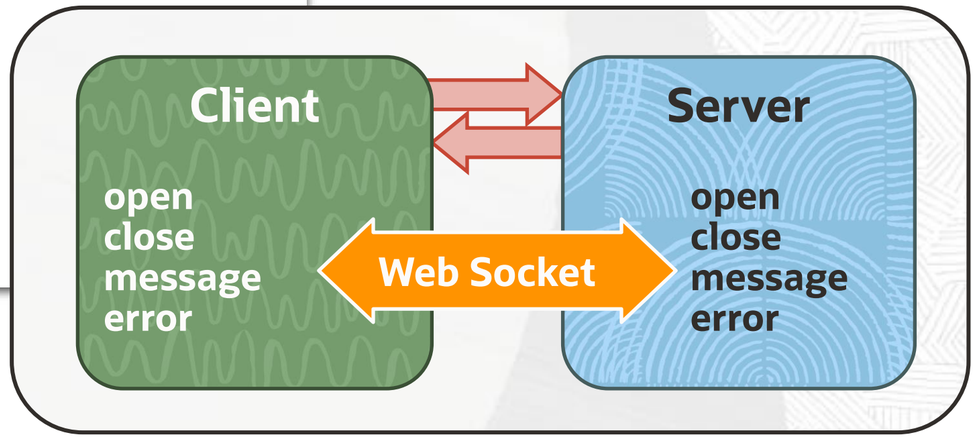


WebSocket Life Cycle

Both client and server endpoints provide symmetrical capabilities handled via the following events:

- `open` –Triggered when a new connection is opened
- `close` – Triggered when a connection is closed (optional parameters: code and reason)
- `message` – Triggered every time a socket receives a message
- `error` – Triggered when an error occurs
- An event object enables access to socket messages and their context.

```
let socket = new WebSocket("ws://dukelabs.org/chat");
socket.addEventListner("message", onMessage);
socket.addEventListner("error", onError);
socket.addEventListner("close", onClose);
function onMessage(evt) { }
function onError(evt) { }
function onClose(code, reason) { }
```



Send and Receive Messages Via the WebSocket

- Send a message through the socket.
- Receive a message through the socket.
- Message data can be in any format (for example, JSON).

```
function handleMessages(evt) {
  let message = JSON.parse(evt.data);
  // handle message object
}

socket.send(JSON.stringify(obj));
```


Stateless Versus Stateful

Stateless: New context is established for each request from a client

Stateful: The context of server interactions is preserved for the lifetime of each client

HTTP interactions are stateless by default.

- A new connection is opened and closed in every request/response cycle.
- Stateful behaviors can be emulated by saving context information between calls.

WebSocket interactions are stateful.

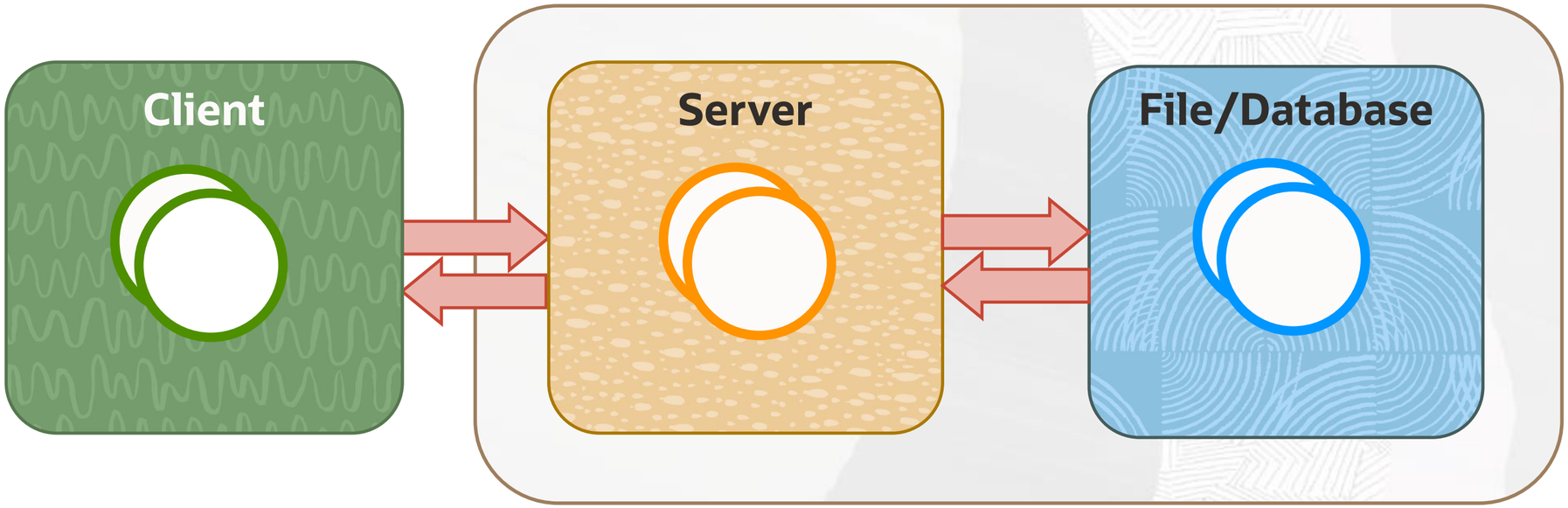
- Connection context is retained for the lifetime of a session.

❖ **Note:** Stateless applications scale better than stateful.

Manage Application State

Preserving context between calls:

- Server-side – WebSocket Context Objects, HTTP Sessions, or File/Database storage
- Client-side – Parameters, Headers, Cookies, and Local and Session storage



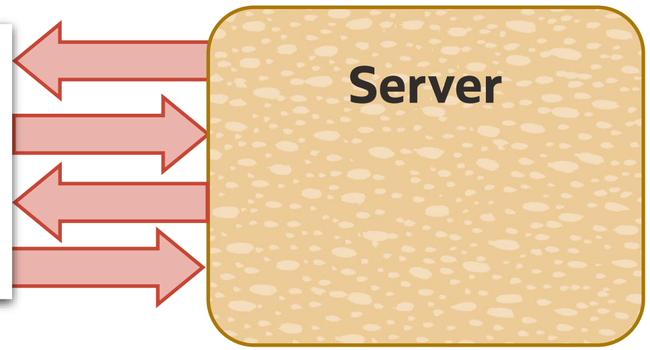
❖ **Note:** This course focuses on client-side options.

Retain State Using Parameters or Headers

To preserve data between HTTP calls:

- Place data into hidden fields
- Send this data to the server in the next request
  - Automatically send fields as request parameters when the form is submitted.
  - Programmatically extract data from fields and send it to the server as parameters or headers.

```
<form action='server-url' method='post'>
  <input id ='x1' type='hidden' name='x1'>
  <input type='submit'>
</form>
```




```
let data = JSON.stringify({x1:document.getElementById('x1')});
// send data as REST parameters or headers
```

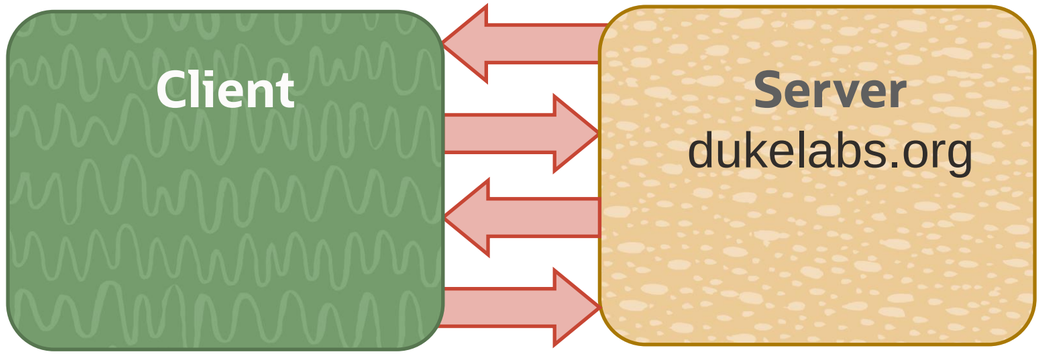
❖ **Note:** HTTP headers are not suitable for transmitting large objects.

HTTP Cookies

An HTTP Cookie is a small piece of data (a name-value pair) retained by the browser. The browser sends to the server all cookies that match the required properties:

- Domain – Host server associated with this cookie
- Path – URL path within the server associated with this cookie
- Expires – Time when cookie expires, otherwise it is retained for the lifetime of a browser session
- SameSite – Restricts sending cookies to CORS services
- Secure – Only allows you to transmit this cookie over HTTPs
- HttpOnly – Makes a cookie not accessible to JavaScript

```
Experiment_id=101;
Domain=dukelabs.org;
Path=/experiments
Expires=Fri, 12 Aug 2022 18:32:51 BST;
SameSite=None;
Secure;
```



Set and Get HTTP Cookies in JavaScript

- Assigning a value to the `document.cookie` property appends a new cookie.
- Retrieving a value of `document.cookie` returns all cookies.
- All cookies are concatenated into a single string.

```
// append cookies
document.cookie = "experiment_id=101; SameSite=None; Secure";
document.cookie = "descrption=Measure things; SameSite=None; Secure";

// get all cookies as a single string
let allCookies = document.cookie;
// parse cookies string and present as an array
let cookiesArray = document.cookie.split("; ");
for (let cookie of cookiesArray){
  console.log(cookie);
}
```

❖ **Note:** Cookies are limited in size and may cause performance and security issues.

Local and Session Web Storage

Store up to 5Mb of data on the client:

- Local store keeps data as long as required
- Session store keeps data only while the browser tab is opened
- Both stores provide identical operations

```
let store = window.localStorage;
// let store = window.sessionStorage;
let weight = { unit:"kg", value:42 };
store.setItem("w",weight);
store.getItem("w");
for (let i = 0; i < store.length; i++) {
  store.getItem(store.key(i));
}
store.removeItem("w");
store.clear();
```

- Put key-value pair
- Get value for a key
- Identify number of objects
- Get key using an index
- Remove a value
- Clear entire store

❖ **Note:** Web Storage has better security and performance than Cookies.

Summary

- Produce and parse JSON
- Invoke REST Services
  - Old-style AJAX client
  - Modern style promises, async/await client
- Implement WebSockets interactions
- Manage application state with parameters, headers, cookies, and local and session storage

Practice 8

- Invoke REST Service
- Create a WebSocket Client
- Cache Values in Local Storage

ORACLE

9

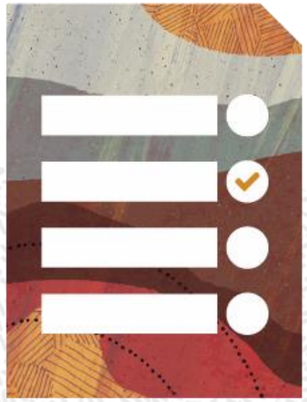
JavaScript Variants, APIs, Frameworks, and the Cloud Environment

Objectives

- JavaScript variants and alternatives
- JavaScript libraries and frameworks
- Server-side JavaScript
- JavaScript and Oracle Cloud Application Development

Agenda

- JavaScript Variants and Alternatives
- JavaScript Libraries and Frameworks
- Server-side JavaScript
- JavaScript and Oracle Cloud Application Development



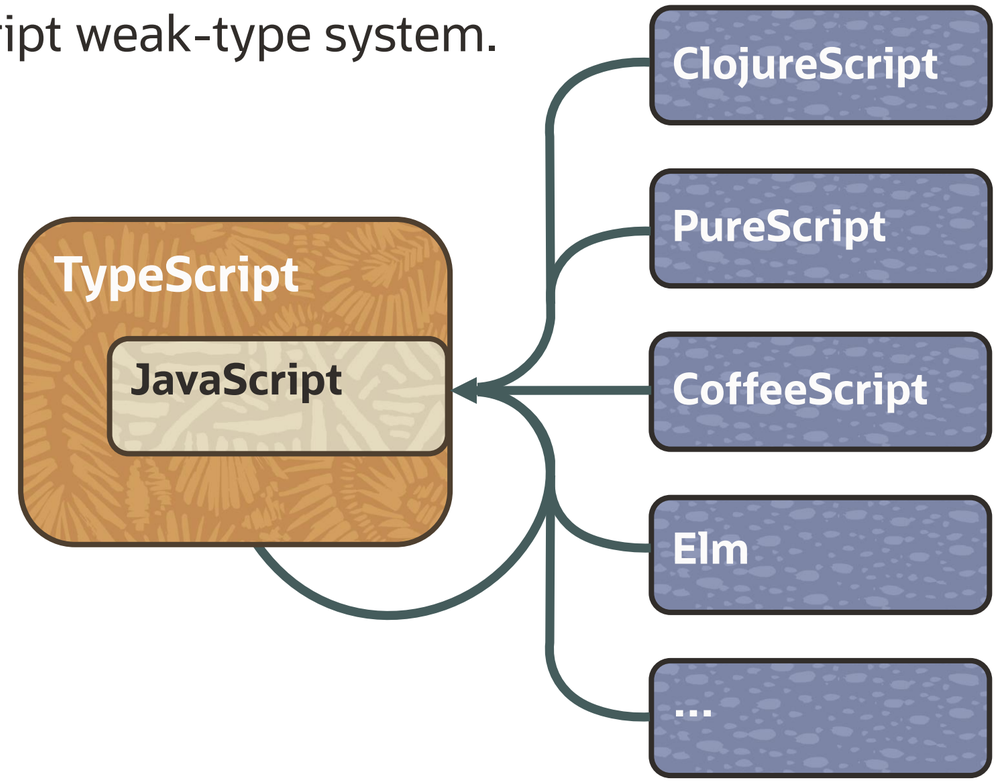
JavaScript Variants and Alternatives

Various programming languages can be translated into JavaScript.

- Use a language of your choice.
- Translate code to JavaScript.
- Execute in a browser, like any other JavaScript application.

TypeScript is a superset of JavaScript.

- It uses static types to address issues of the JavaScript weak-type system.
- TypeScript validates types of variables.
- It can be translated into JavaScript.



TypeScript Type Enforcement

- Explicit type declarations, design-time validation, and restricted type reassignments
- Type can be determined implicitly based on a value
- Encapsulation using `private` variables and methods

```
let a = "Hi";
a = 42;
class Measurement {
  private unit: string;
  private value: number;
  constructor (unit: string, value: number) { }
}
let m1 = new Measurement("kg", 42);
m1.id = 101;
```

TypeScript Extra Types

- `any` – Switch off type enforcement for a given object.
- `void` – Define a function that does not return a value.
- `never` – Define a function that never returns.
- union – Enlist a number of allowed types.
- intersection – Combine different types into one.

```
class Measurement {
  private unit: "m" | "kg";
  private value: number | string;
  converter: Weight & Length;
  function print(): void { console.log(unit+" "+value); }
  function bad(): never { throw new Error(); }
}
let m1 = new Measurement("kg", 42);
m1.converter.kgToLb(42);
m1.converter.mToFt(42);
```

```
class Weight {
  kgToLb(value: number) {
    return number*2.2;
}}
```

```
class Length {
  mToFt(value: number) {
    return number*3.3;
}}
```

```
let x: any = { b:"xyz" };
x.a = 101;
```

TypeScript Enumerations

- An `enum` construct defines a type that is restricted with explicitly enumerated fixed values.
- It may automatically initialize values starting from a specified value.

```
enum Unit {
   s = "Second",
   M = "Meter",
   Kg = "Kilogram",
   A = "Ampere",
   K = "Kelvin",
   mol = "Mole",
   cd = "Candela"
}
```

```
enum ErrorMargin {
 low = 1,
 medium,
 high
}
```

```
class Measurement {
  private unit: Unit;
  private margin: ErrorMargin;
  constructor(unit: Unit, margin: ErrorMargin) {
   this.unit = unit;
   this.margin = margin;
  }
}
let m1 = new Measurement(Unit.M, ErrorMargin.low);
```

… And Many More TypeScript Features

TypeScript and JavaScript compatibility

- Reuse existing JavaScript code and libraries in TypeScript applications.
- Invoke TypeScript code from JavaScript applications.

Other features include:

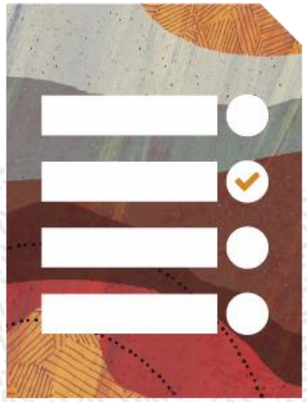
- Generics
- Interfaces

```
function doThings<Type>(value: Type) {
  /* type is to be determined
     upon use of this function */
}
doThings<number>(42);
doThings<string>("acme");
doThings<string>(42);
```

```
interface Format {
  format(): string;
}
class Weight implements Format {
  format():string {
  // implement weight format
  }
}
class Length implements Format {
  format():string {
  // implement length format
  }
}
```

Agenda

- JavaScript Variants and Alternatives
- JavaScript Libraries and Frameworks
- Server-side JavaScript
- JavaScript and Oracle Cloud Application Development



JavaScript Libraries and Frameworks

- jQuery and jQueryUI
- ReactJS
  - JavaScript Syntax Extension (JSX)
- PreactJS, a faster, lightweight version of ReactJS
- RequireJS
- Knockout
- AngularJS
- Oracle JavaScript Extension Toolkit (JET)
  - Based on open-source libraries such as jQuery and PreactJS
  - Supports development in both JavaScript and TypeScript

...and many others

Manage Library References

Include library resources into a page

- One or more JavaScript files
- Other resources, such as CSS files

Reference jQuery and jQueryUI libraries

```
<html>
  <head>
    <link rel="stylesheet" href="lib/jquery-ui.css">
    <script src="lib/jquery.js"></script>
    <script src="lib/jquery-ui.js"></script>
    <script src="my.js"></script>
  </head>
  ...
</html>
```

Use jQuery Selectors and Functions

Use library functionalities.

- Select one or more page elements: `$(selector)` or `jQuery(selector)`
- Selector uses CSS-like syntax
- Perform actions with selected elements:
  - Associate elements with an event handler
  - Modify elements
  - Run animations

```
<html><head>...</head>
<body>
  <div id='x1'>Click me!</div>
  <div>Hide me!</div>
</body></html>
```

```
window.onload => () {
  $('#x1').on('click', ShowHide);
}
function ShowHide() {
  $('#x1').animate({
    backgroundColor: "#FFFF00",
    color: "#0000FF",
    fontSize: 20
    }, 1000 );
  $('#x1').next().toggle();
}
```

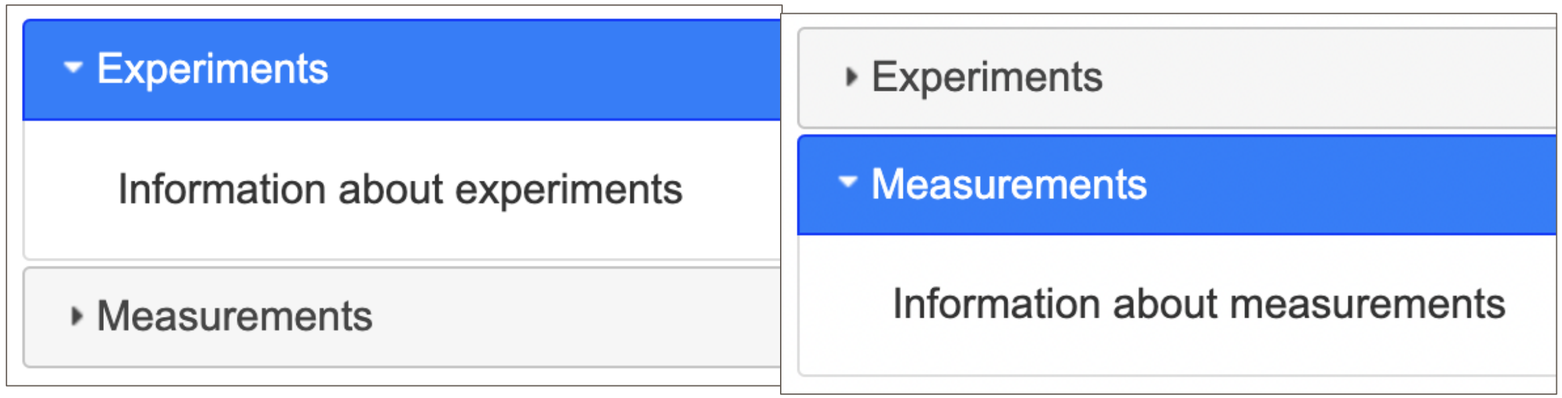
Use jQueryUI Widgets

- A jQueryUI widget is a UI component with predefined appearance and behaviors.
- A number of different widgets are provided by the jQueryUI library.

For example, an **accordion** widget turns a group of dividers into a set of switchable tabs:

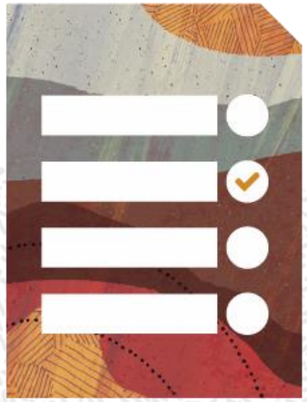
```
<div id="x1">
  <h3>Experiments</h3>
  <div>Information about experiments</div>
  <h3>Measurements</h3>
  <div>Information about measurements</div>
</div>
```

```
window.onload => () {
  $('#x1').accordion();
}
```



Agenda

- JavaScript Variants and Alternatives
- JavaScript Libraries and Frameworks
- Server-side JavaScript
- JavaScript and Oracle Cloud Application Development



Implement Server-Side JavaScript with Node.js

Node.js is an open-source and cross-platform JavaScript runtime environment.

- Runs on the Google V8 JavaScript engine
- Handles all requests in a single process, without creating a new thread per request
- Implements asynchronous IO to facilitate nonblocking code execution

```
// define a server instance
const server = require('http').createServer((req, res) => {
  // describe what server should be doing
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello World\n');
});
// launch server
server.listen(80, 'dukelabs.org', () => {
  console.log("Server is running");
});
```


Manage Node.js Projects

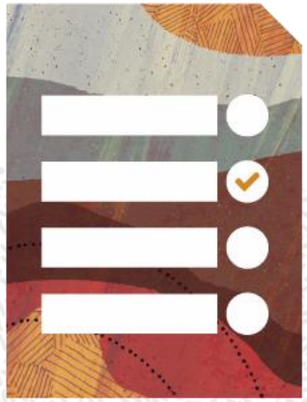
- A project and its dependencies are described in the `package.json` file.
- Project dependencies are managed with the `npm` command line tool.

```
{
  "name": "DukeLabs",
  "author": "Oracle",
  "description": "DukeLabs Services",
  "version": "1.2.3",
  "dependencies": {
     "typescript": "next"
  },
  "scripts": {
    "start": "node dukeserver.js"
  }
  "main": "duke.js"
}
```

```
npm install
npm update
npm run start
```


Agenda

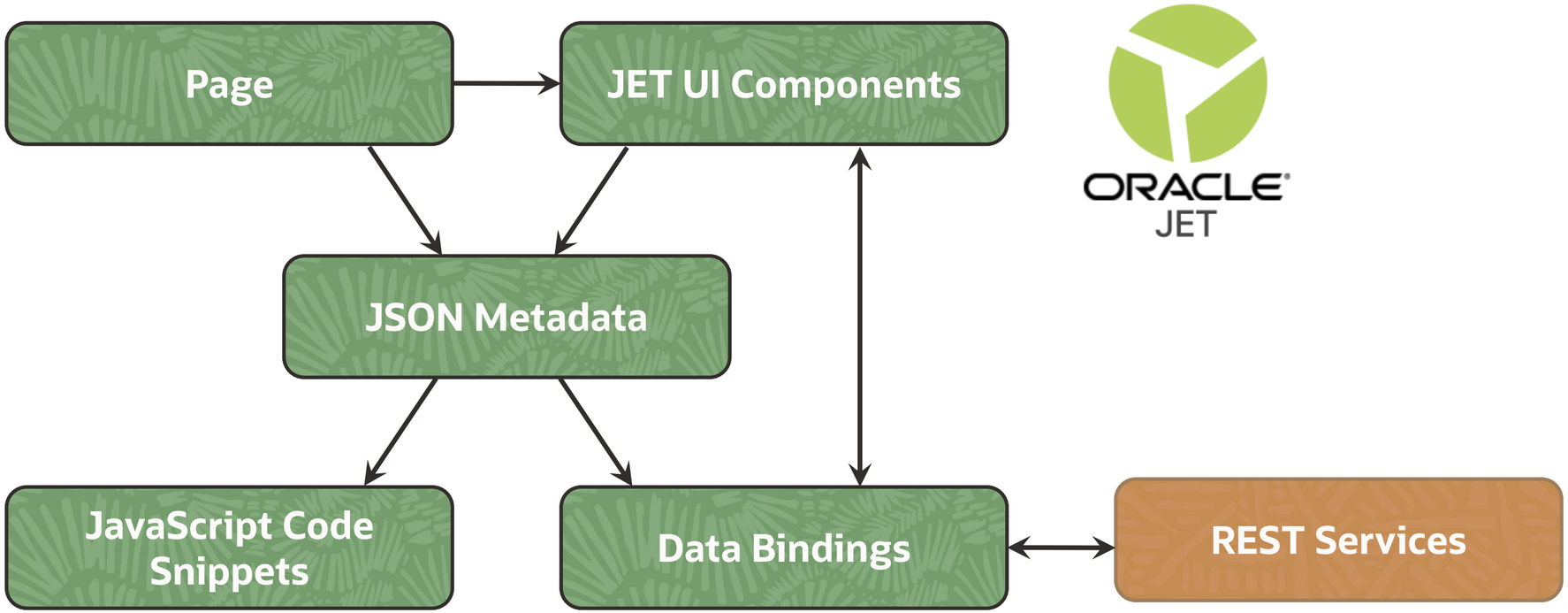
- JavaScript Variants and Alternatives
- JavaScript Libraries and Frameworks
- Server-side JavaScript
- JavaScript and Oracle Cloud Application Development



JavaScript and Oracle Cloud Application Development

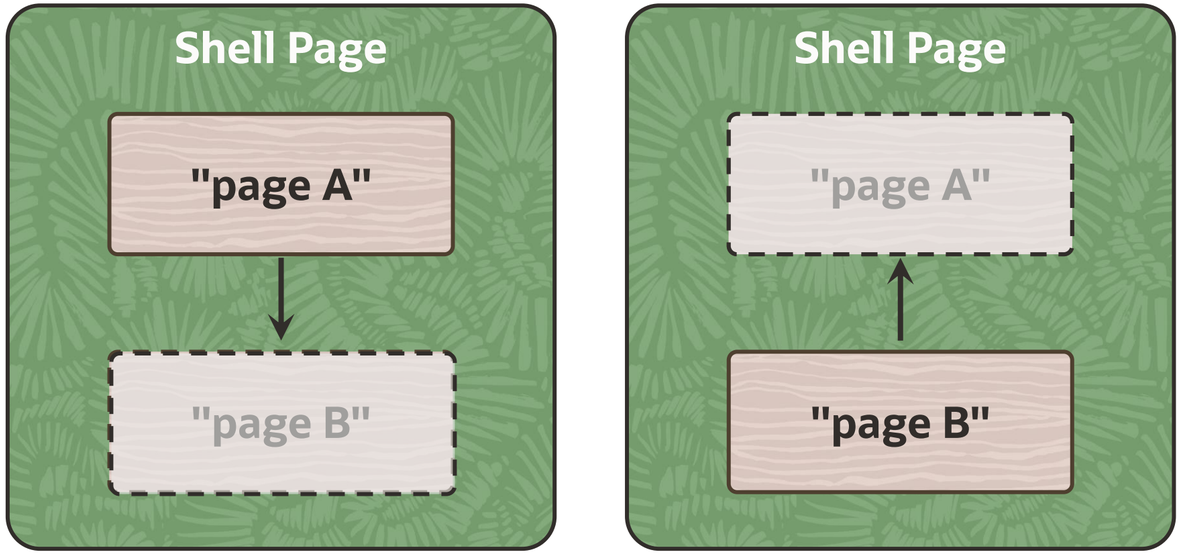
Visual Builder Studio and Visual Builder

- A collection of collaborative development and project management tools
- Designed to produce responsive (browser and mobile) applications
- Implementing application with HTML5, JavaScript, CSS, and Oracle JET framework
- Utilizes a single-page application UI design model



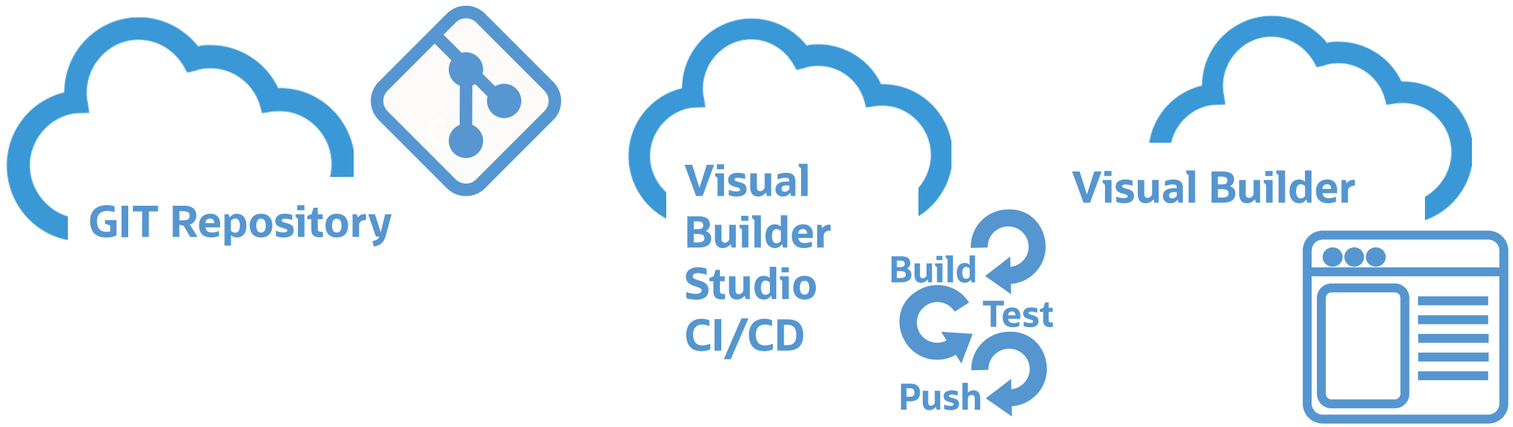
Single-Page Application Design

- A shell page is the only physical page in the application.
- It contains all other components, some of which are hidden from view.
- Different components are shown and hidden based on user actions *(this feels like navigation between pages).*
- A client downloads a single page that contains all other "pages" inside.



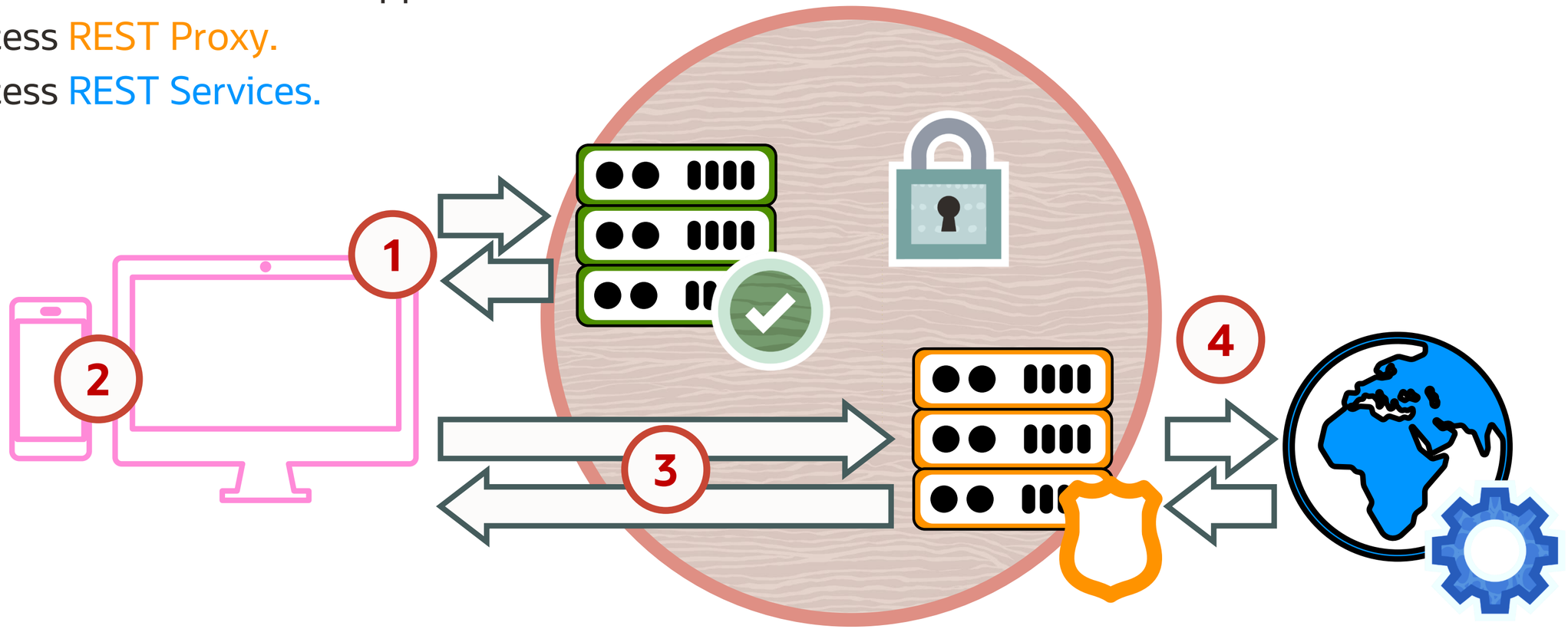
Visual Builder Studio Project Management

- Accelerated application development with Visual Builder
- Oracle Cloud Applications extensions and configuration
- Source control with GIT integration
- Team collaboration and CI/CD automation
- Application hosting



Visual Builder Application Runtime

1. Acquire application code from the Cloud Hosting Server.
2. Initialize and render the application on a Client.
3. Access REST Proxy.
4. Access REST Services.



❖ **Note:** Access to the application and services is protected by IDCS.

Summary

- JavaScript variants and alternatives
- JavaScript libraries and frameworks
- Server-side JavaScript
- JavaScript and Oracle Cloud Application Development